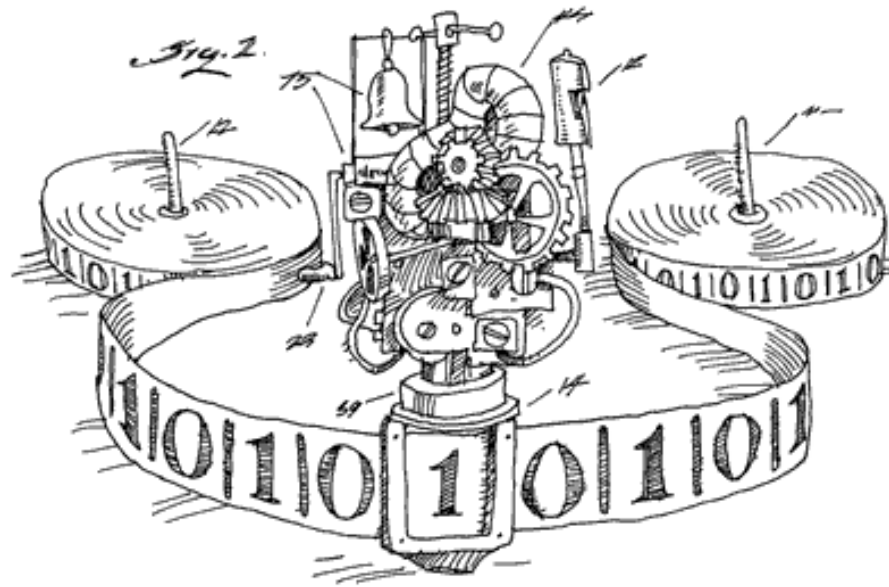


Turing Machines



Representing problems

A **decision problem** can be thought of as

$$f : \Sigma^* \rightarrow \{\text{No}, \text{Yes}\}$$

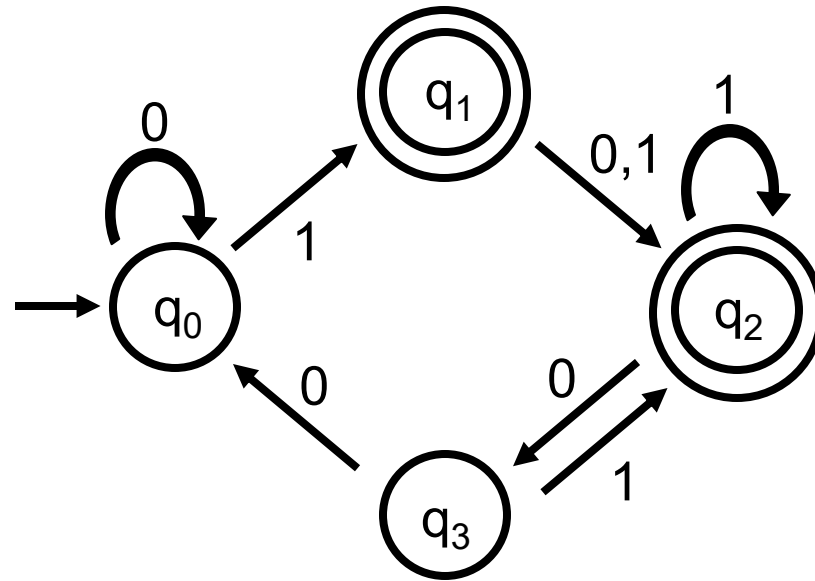
or equivalently as a **language** $L \subseteq \Sigma^*$

where $L = \{x \in \Sigma^* : f(x) = \text{Yes}\}$.

$$\text{Equivalently, } f(x) = \begin{cases} \text{Yes} & \text{if } x \in L \\ \text{No} & \text{if } x \notin L \end{cases}$$

E.g.: $L_{\text{PALINDROME}} = \{x \in \Sigma^* : x \text{ is a palindrome}\}$

Review: DFAs



Recall: There are simple languages like $L = \{x^k y^k : k \geq 0\}$ that no DFA can decide!

Review: DFAs Deciding Languages

Let M be a DFA, using alphabet Σ .

M **accepts** some strings in Σ^* and **rejects** the rest.

Definition: $L(M) = \{x \in \Sigma^* : M \text{ accepts } x\}$

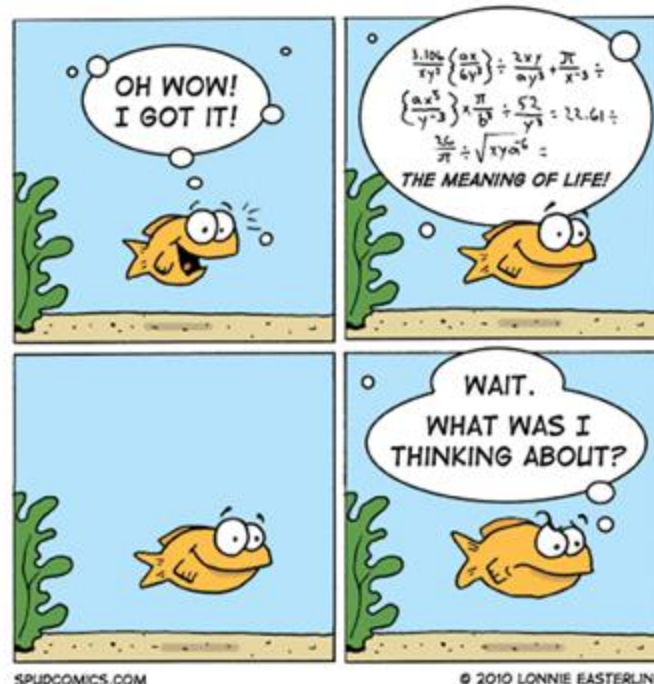
Called the “language decided by M ”.

If L is a language,

we say M **decides** L if $L(M) = L$.

Intuitive reason why DFAs cannot decide many languages:

Fixed amount of memory!



SPUDCOMICS.COM

© 2010 LONNIE EASTERLING

THE TRAGEDY OF A THREE SECOND MEMORY

A DFA models a computer with a fixed amount of memory
(think of the states as “memory”)

Real computers are only as powerful as DFAs.

Your laptop also cannot decide the language $L = \{x^k y^k : k \geq 0\}$.

But, in our definition of a “computer”, we don’t want to artificially constrain ourselves to a fixed amount of memory.

Instead, we want our definition of a “computer” to model the situation where you can always buy more memory if you need it.

That’s where **Turing Machines** come in.

But first, some history...

Some History: Hilbert's 10th Problem (1900)

Consider a multivariate polynomial with integer coefficients set equal to 0

E.g. $4x^2y^3 - 2x^4z^5 + x^8 = 0$.



“Devise a *process according to which it can be determined in a finite number of operations* whether it has an integer solution.”

$x^2 - 4 = 0 \rightarrow$ yes

$n^2 - 2 = 0 \rightarrow$ no

$4x^2y^3 - 2x^4z^5 + x^8 = 0 \rightarrow ??$

This is asking for an **algorithm**.

Entscheidungsproblem (1928)

Given a statement in first-order logic.

Devise an “*effectively calculable procedure*”
that determines if it is valid.

“All kittens are cats, all cats are cute, therefore all kittens are cute” -> true

“All kittens are cats, all kittens are cute, therefore all cats are cute” -> false

“All even numbers greater than 2 are the sum of two primes” -> ?????

Again, this is asking for an **algorithm**.

Mathematicians began to think hard about a
formal definition for “**algorithm**”.



David Hilbert



Wilhelm Ackermann

Gödel (1934):

Discusses some ideas for mathematical definitions of computation procedures, but isn't confident what's a good definition.



Church (1936):

Invents **lambda calculus**, essentially claims it should be the definition of “algorithm”.



Gödel, Post (1936):

Arguments that Church's claim is not justified.



Meanwhile... in New Jersey... a certain British grad student, unaware of all these debates...

Alan Turing (1936, age 22):

Describes a new model of computation,
now known as the **Turing Machine**TM



Gödel, Kleene, and even Church:

“Um, he nailed it. Game over, ‘algorithm’ defined.”

1937: Turing proves **TM's $\equiv$ lambda calculus**

Church–Turing Thesis

“Any natural notion of being ‘algorithmically computable’ is captured by Turing Machines.”

This is not a theorem.

Is it... *... a hypothesis?*

... a definition?

... a philosophical statement?

Well, in any case, everyone seems to believe it.

Anything that can be computed by Python, C++, a quantum computer, LaTeX, pseudocode, AI, etc. can be computed by a Turing Machine.

Entscheidungsproblem: Devise an **algorithm** (Turing Machine) that, given a statement in first-order logic, determines if it's valid.



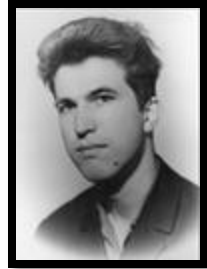
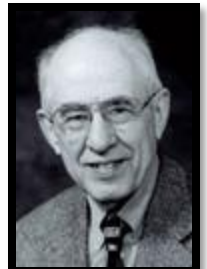
There is no such algorithm! (1936)

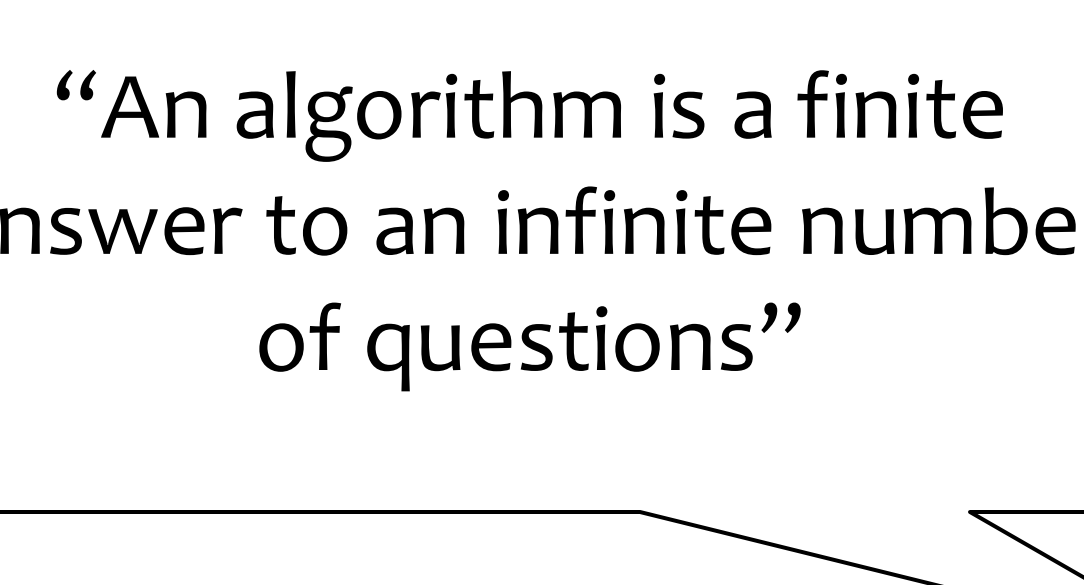


Hilbert's 10th Problem: Devise an **algorithm** (Turing Machine) that, given a multivariate polynomial with integer coefficients, determines if it has an integer root.



There is no such algorithm! (1970)



A large, white speech bubble with a black outline and rounded corners. It has a small tail pointing towards the bottom right corner. Inside the bubble, a quote is written in a black, sans-serif font.

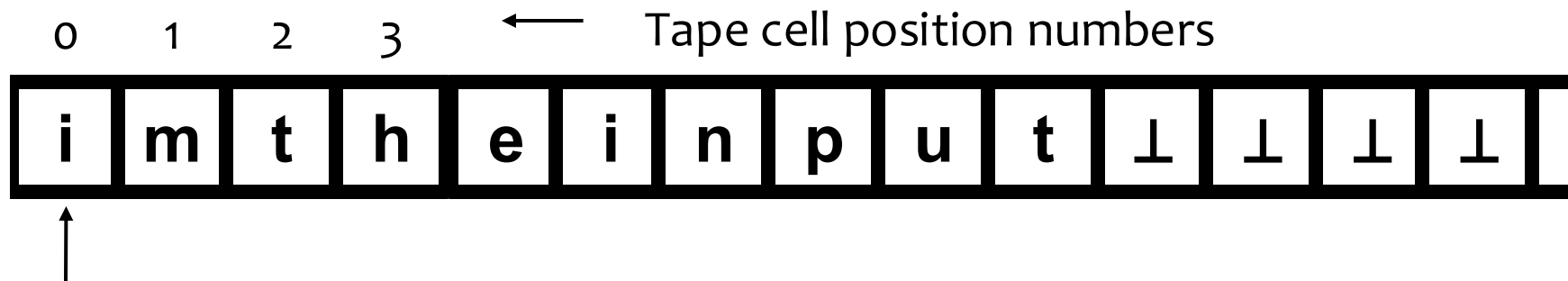
“An algorithm is a finite answer to an infinite number of questions”

$$\begin{array}{l} \cancel{x}^2 + 0 = 0 \Rightarrow \cancel{N} \\ \sim N \\ \sim Y \\ \sim N \\ \sim N \\ \sim Y \\ \vdots \\ \text{infinitely} \end{array}$$


Stephen Cole Kleene

What is a Turing Machine?

Memory of a Turing Machine: an infinite “tape” of cells:



The input from Σ^* is written starting at cell 0.

All other cells contain $\perp$ (blank).

In general, cells contain symbols from a (finite) **tape alphabet** Γ , which must contain Σ , $\perp$, and may have other symbols.

There's a tape pointer (“**head**”), initially at position 0.

What is a Turing Machine?

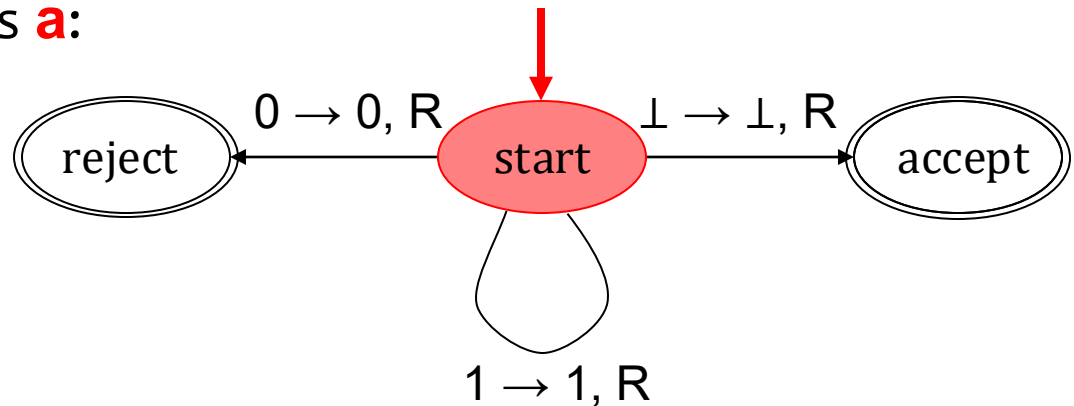
Like a DFA, except each transition specifies:

- what we read from the tape
- what we write to the tape
- whether the head moves *left* (L) or *right* (R)

Specifically: “**a** → **b**, **R**” means:

if the current tape cell is **a**:

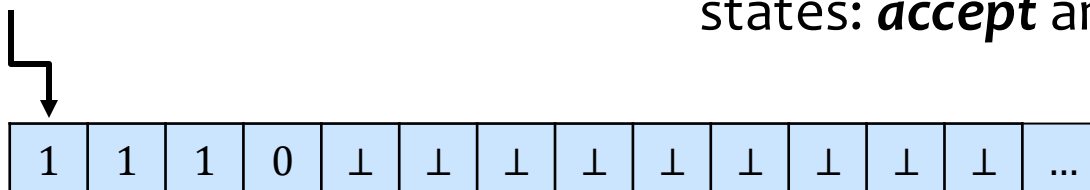
1. write **b**
2. move **right**



Assume that if TM encounters “move left” with head at position 0, the head doesn’t move



INFINITE
TAPE



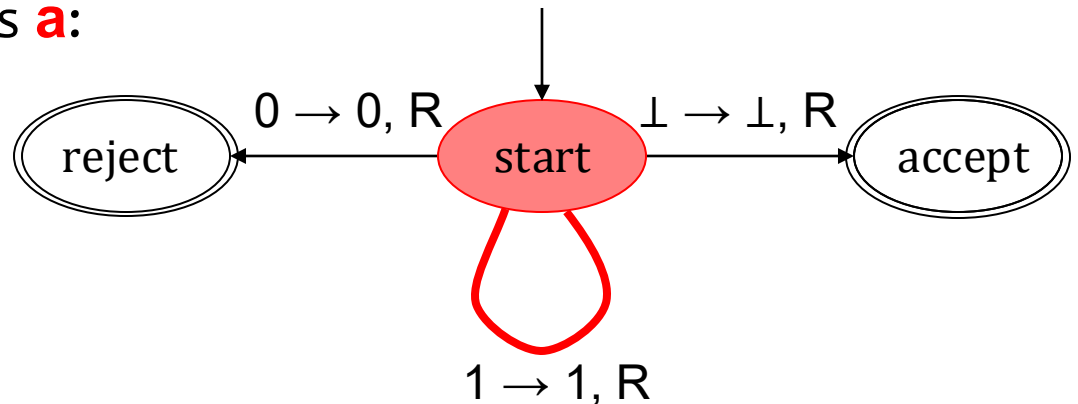
There are two special “termination” states: **accept** and **reject**.

- what we read from the tape

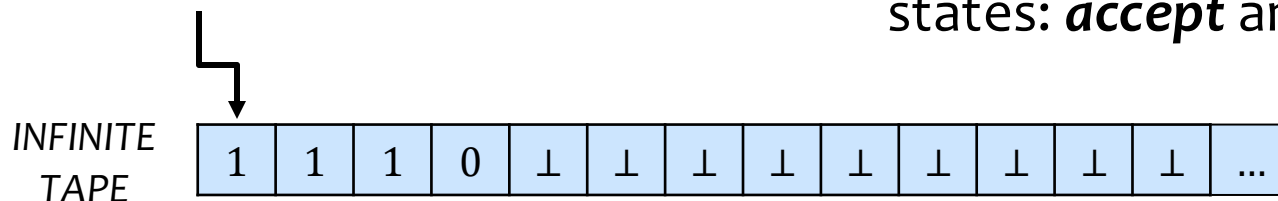
- Specifically: “ **$a \rightarrow b, R$** ” means:

if the current tape cell is **a**:

1. write **b**



There are two special “termination” states: ***accept*** and ***reject***.



What is a Turing Machine?

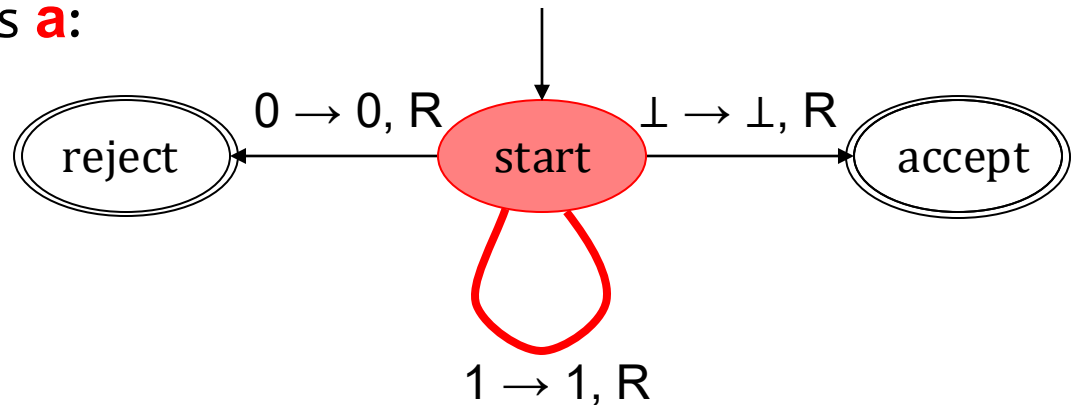
Like a DFA, except each transition specifies:

- what we read from the tape
- what we write to the tape
- whether the head moves *left* (L) or *right* (R)

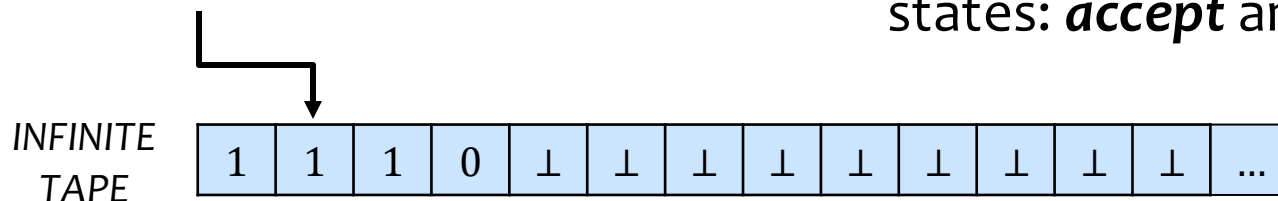
Specifically: “**a** → **b**, **R**” means:

if the current tape cell is **a**:

1. write **b**
2. move **right**



There are two special “termination” states: **accept** and **reject**.



What is a Turing Machine?

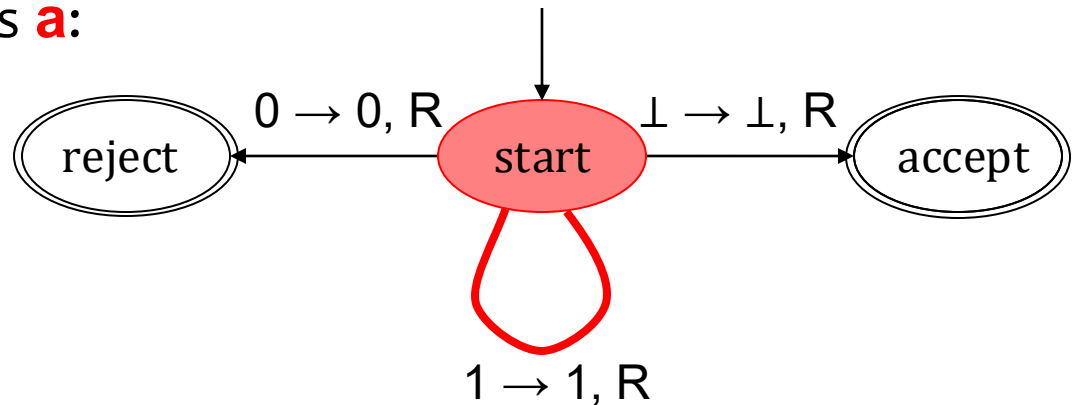
Like a DFA, except each transition specifies:

- what we read from the tape
- what we write to the tape
- whether the head moves *left* (L) or *right* (R)

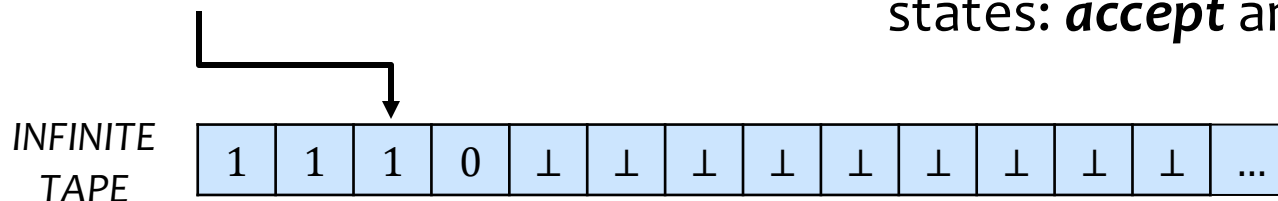
Specifically: “**a** → **b**, **R**” means:

if the current tape cell is **a**:

1. write **b**
2. move **right**



There are two special “termination” states: **accept** and **reject**.



What is a Turing Machine?

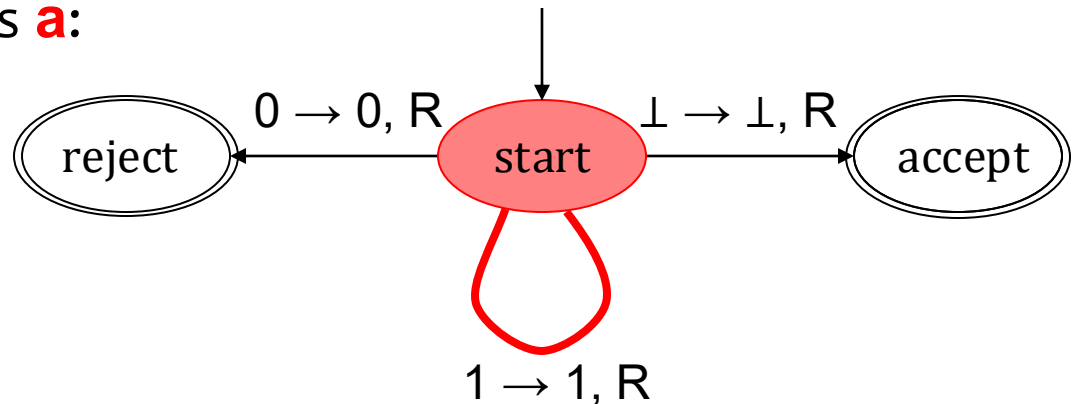
Like a DFA, except each transition specifies:

- what we read from the tape
- what we write to the tape
- whether the head moves *left* (L) or *right* (R)

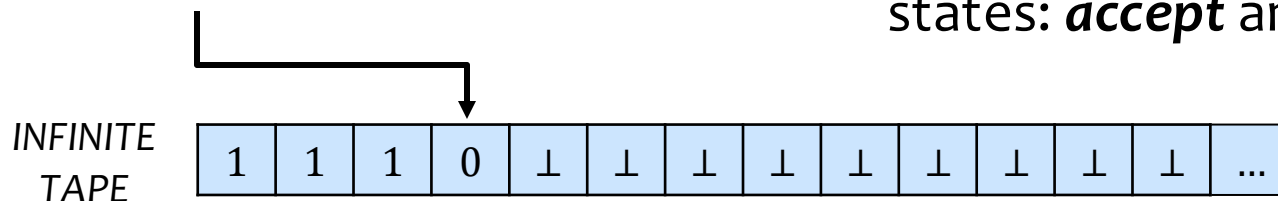
Specifically: “**a** → **b**, **R**” means:

if the current tape cell is **a**:

1. write **b**
2. move **right**



There are two special “termination” states: **accept** and **reject**.

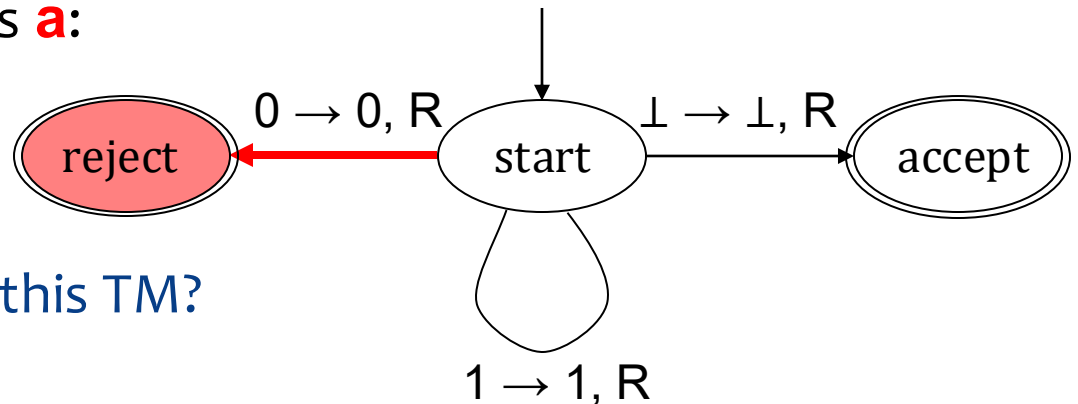


- what we read from the tape

- Specifically: “ **$a \rightarrow b, R$** ” means:

if the current tape cell is **a**:

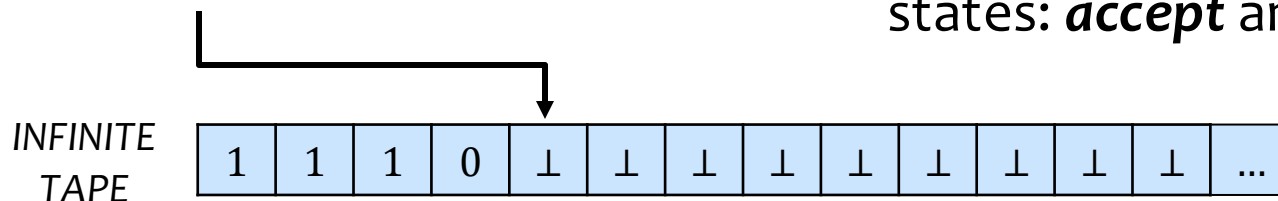
1. write **b**



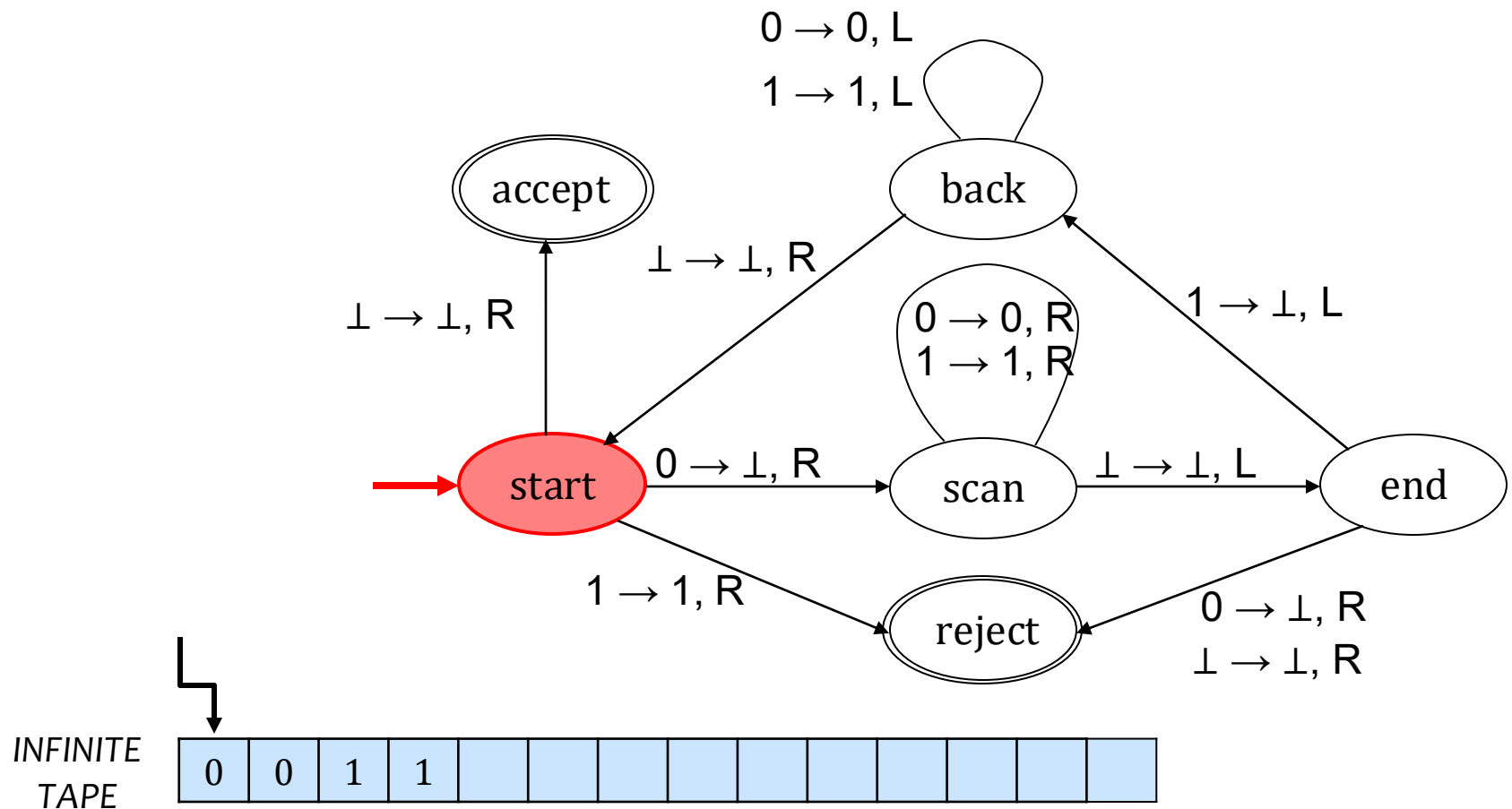
1 *

 1^*

There are two special “termination” states: ***accept*** and ***reject***.

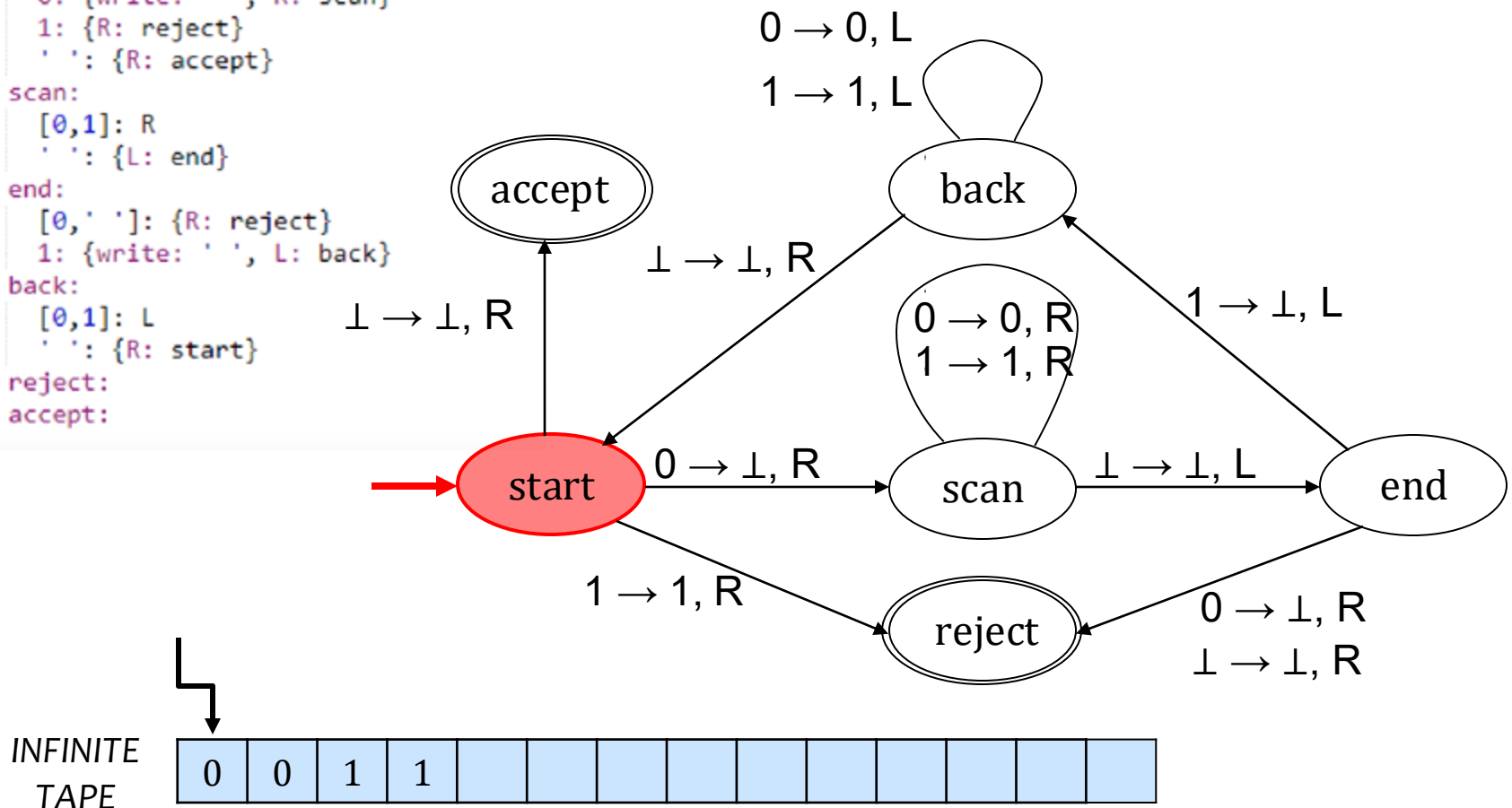


What is a Turing Machine?

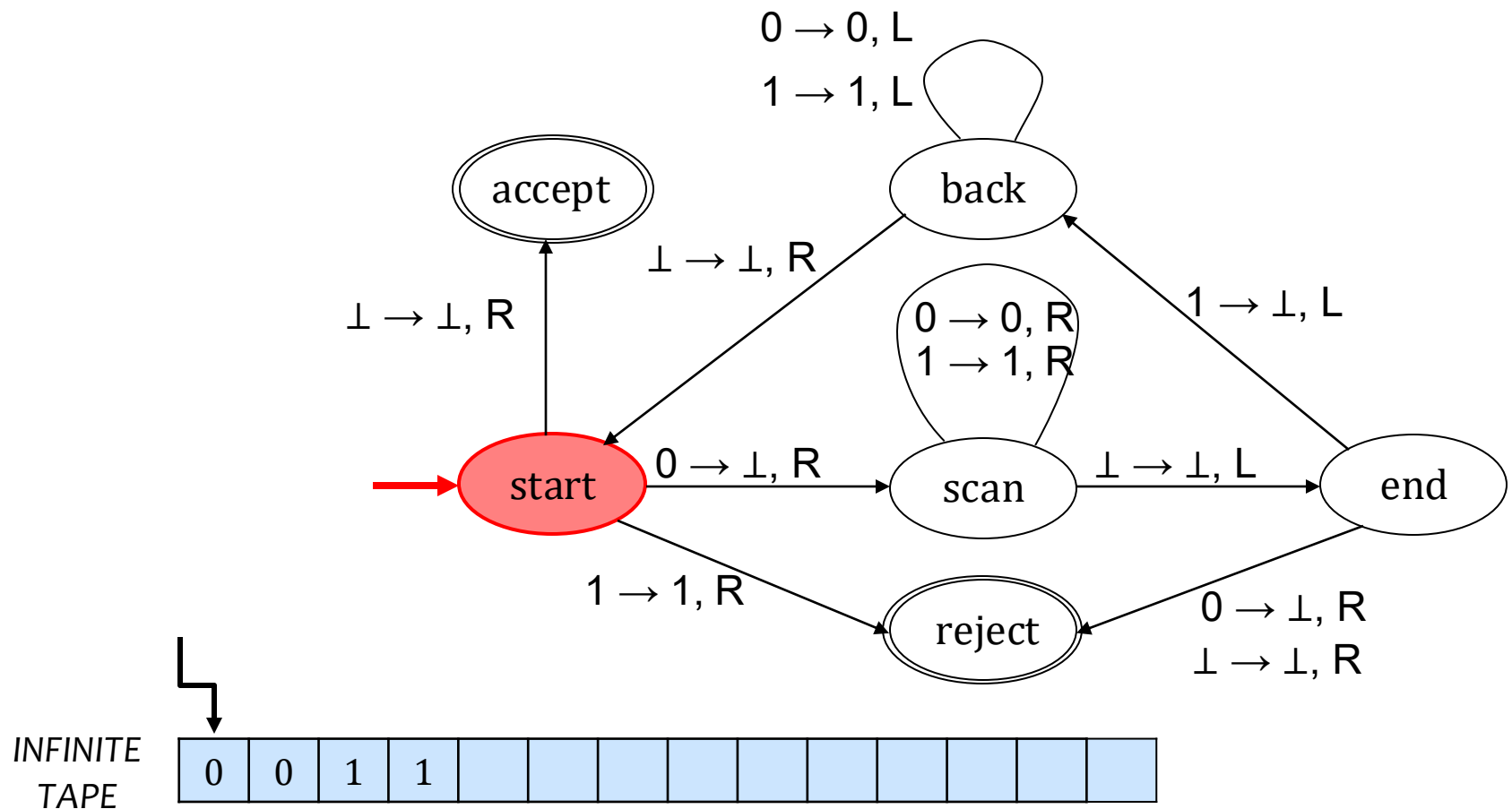


A useful tool: turingmachine.io

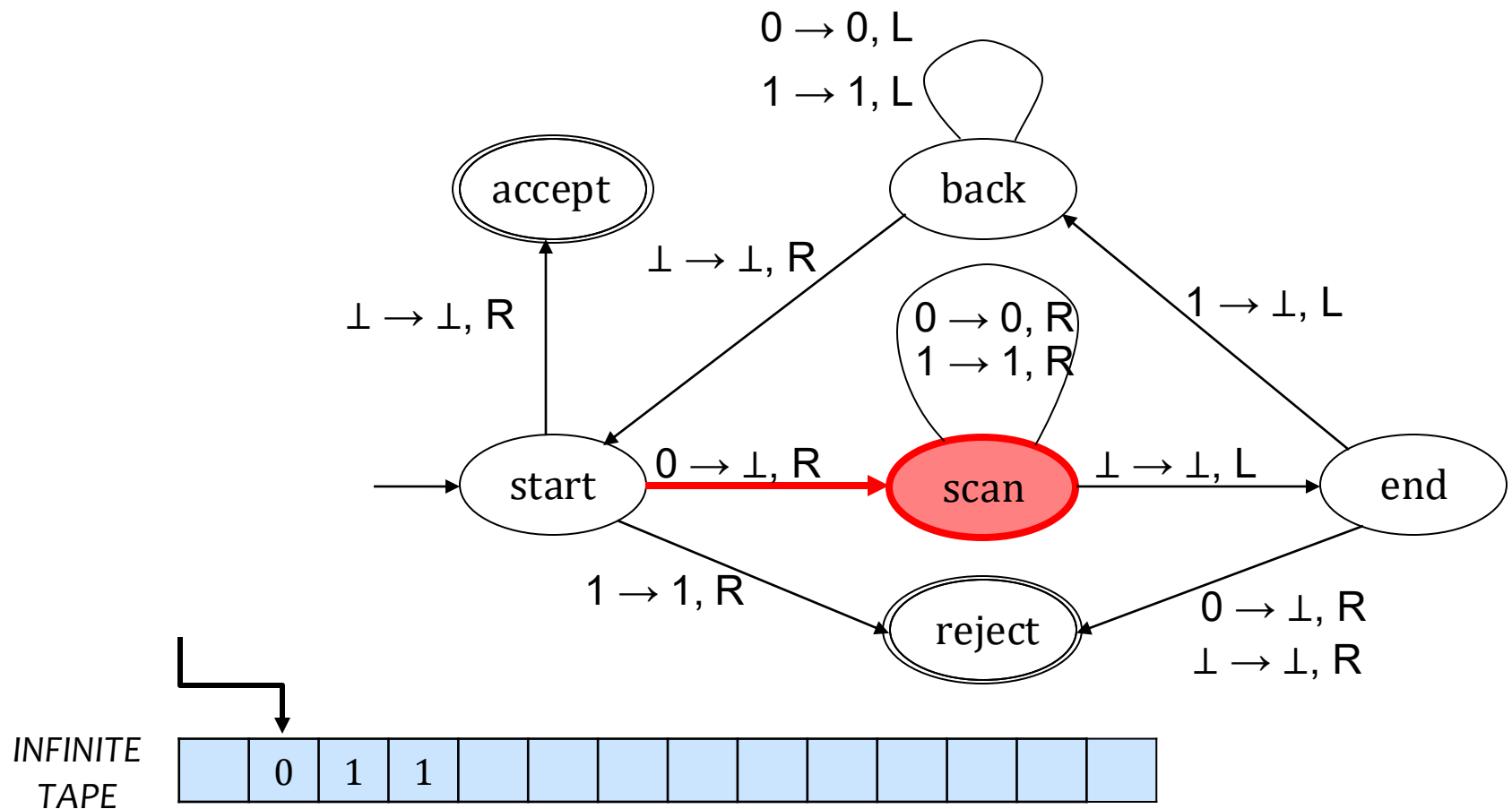
```
3 input: '0011' # try '', '0', '011', '010', '11', '0101'
4 blank: ' '
5 # Blank out the first 0 and last 1 on each pass.
6 # When there are only blanks, all 0's have matched to all 1's.
7 start state: start
8 table:
9   start:
10     0: {write: ' ', R: scan}
11     1: {R: reject}
12     ' ': {R: accept}
13   scan:
14     [0,1]: R
15     ' ': {L: end}
16   end:
17     [0, ' ']: {R: reject}
18     1: {write: ' ', L: back}
19   back:
20     [0,1]: L
21     ' ': {R: start}
22 reject:
23 accept:
```



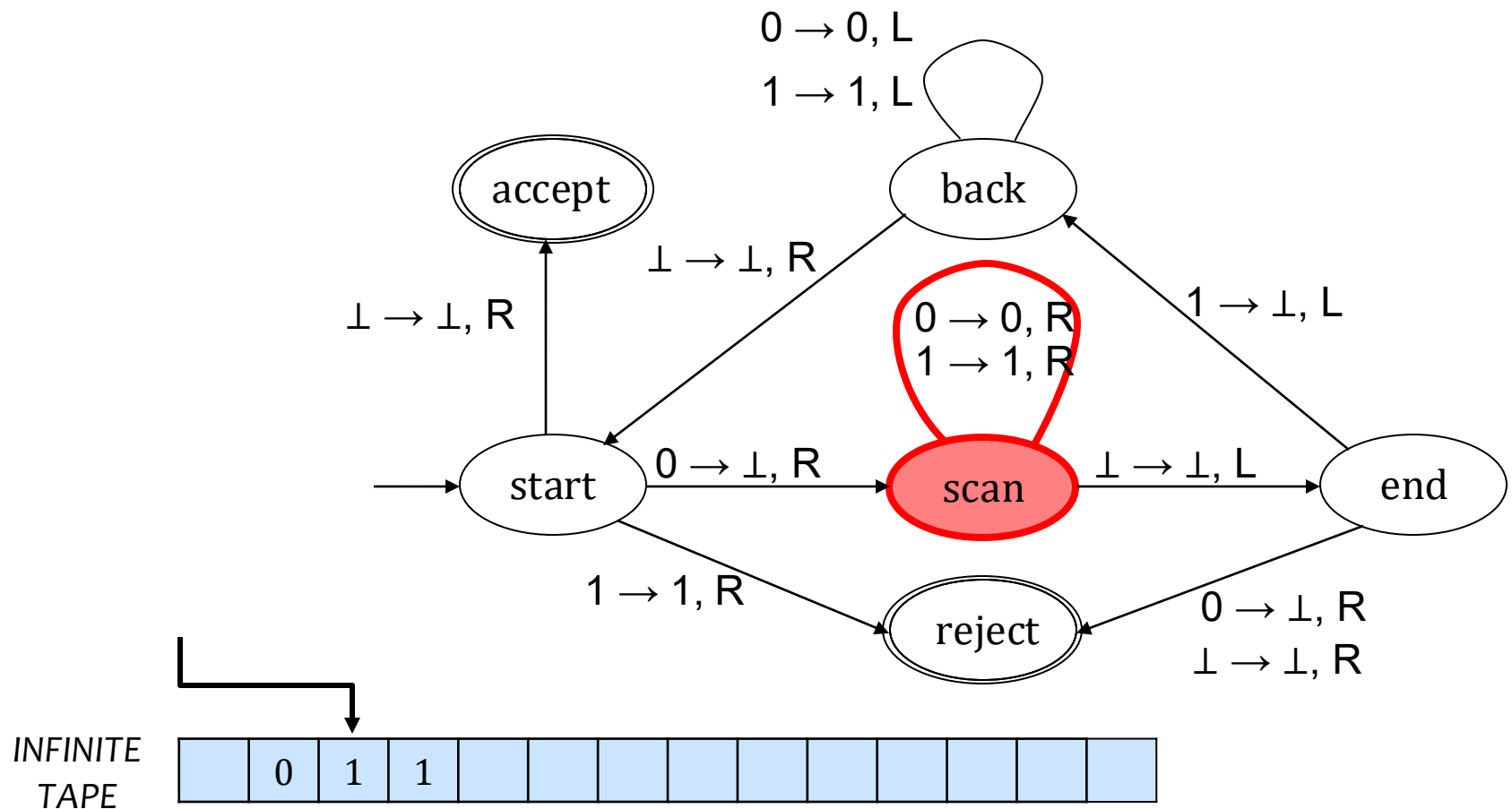
What is a Turing Machine?



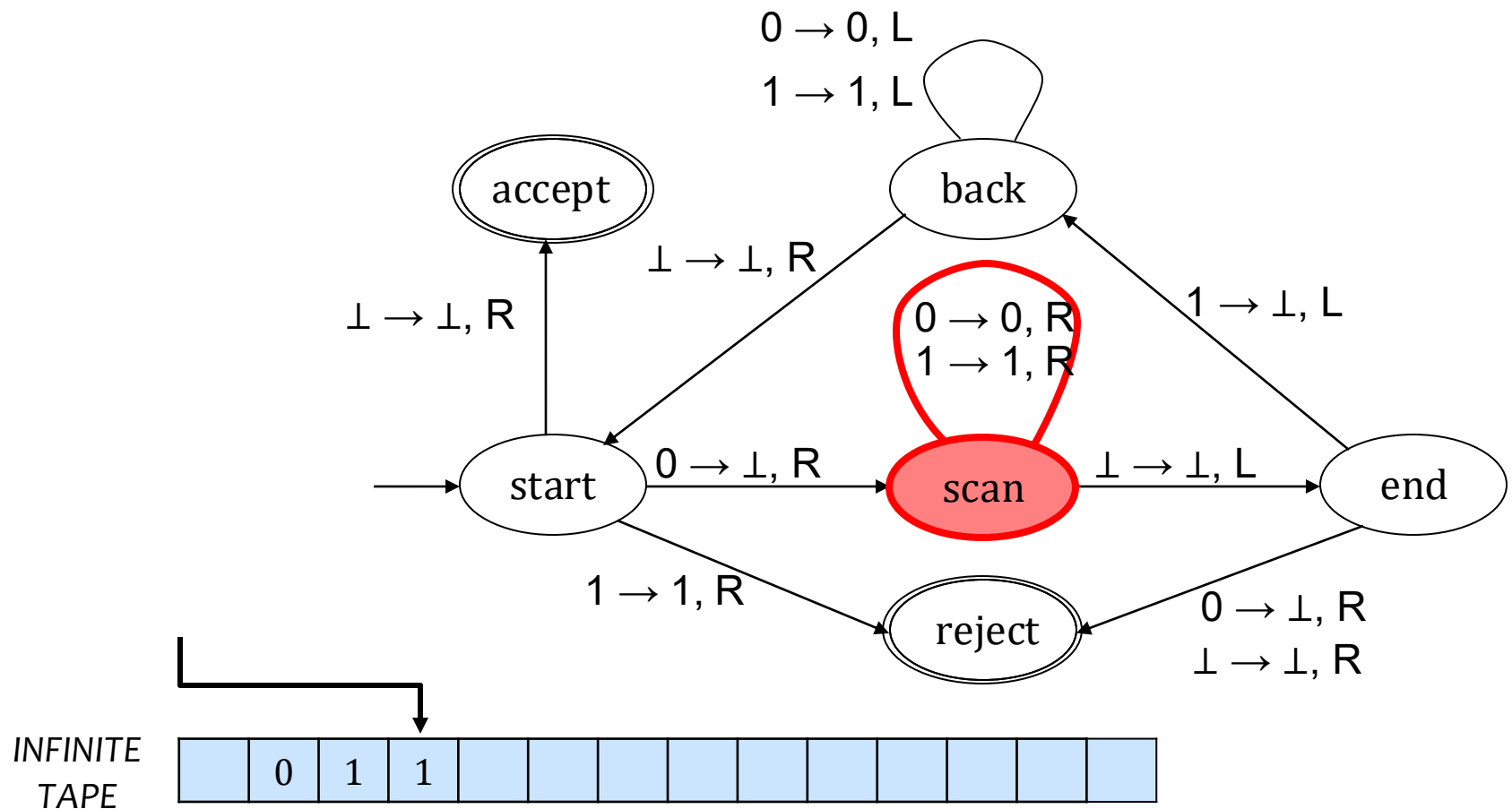
What is a Turing Machine?



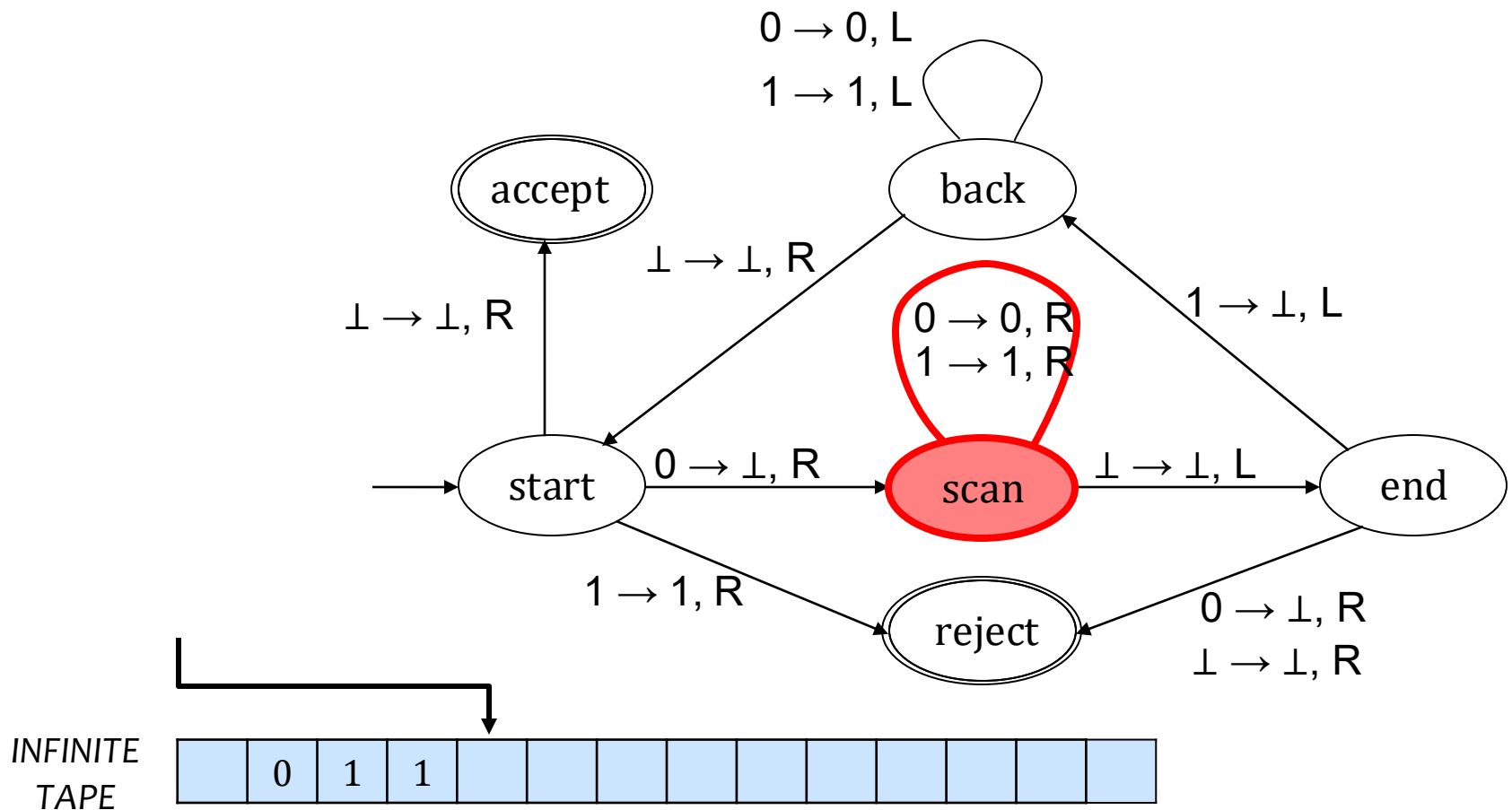
What is a Turing Machine?

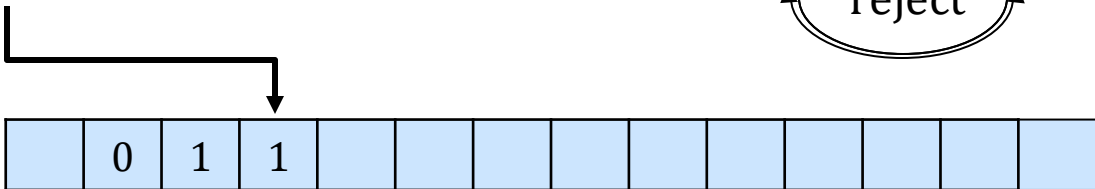


What is a Turing Machine?

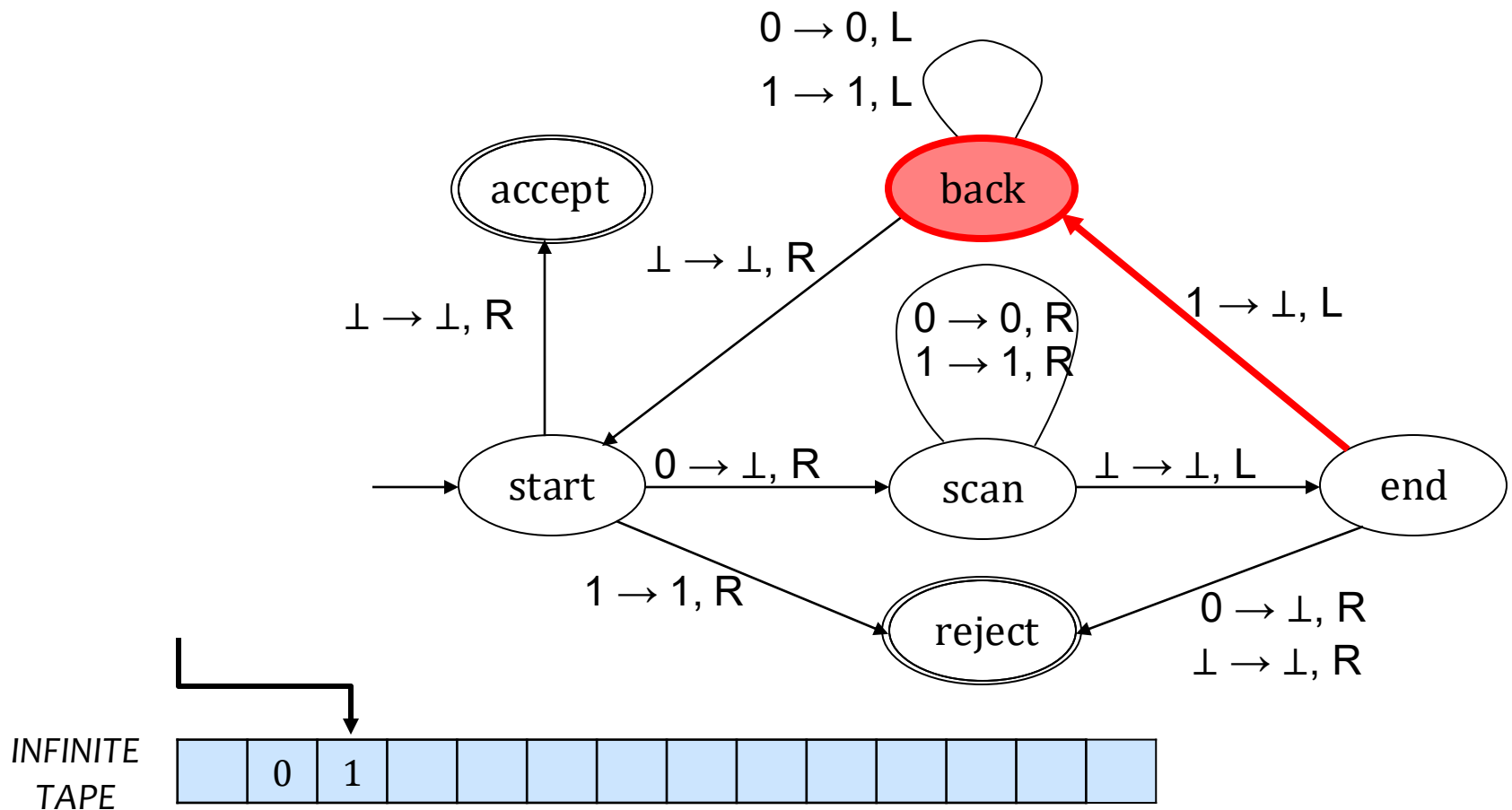


What is a Turing Machine?

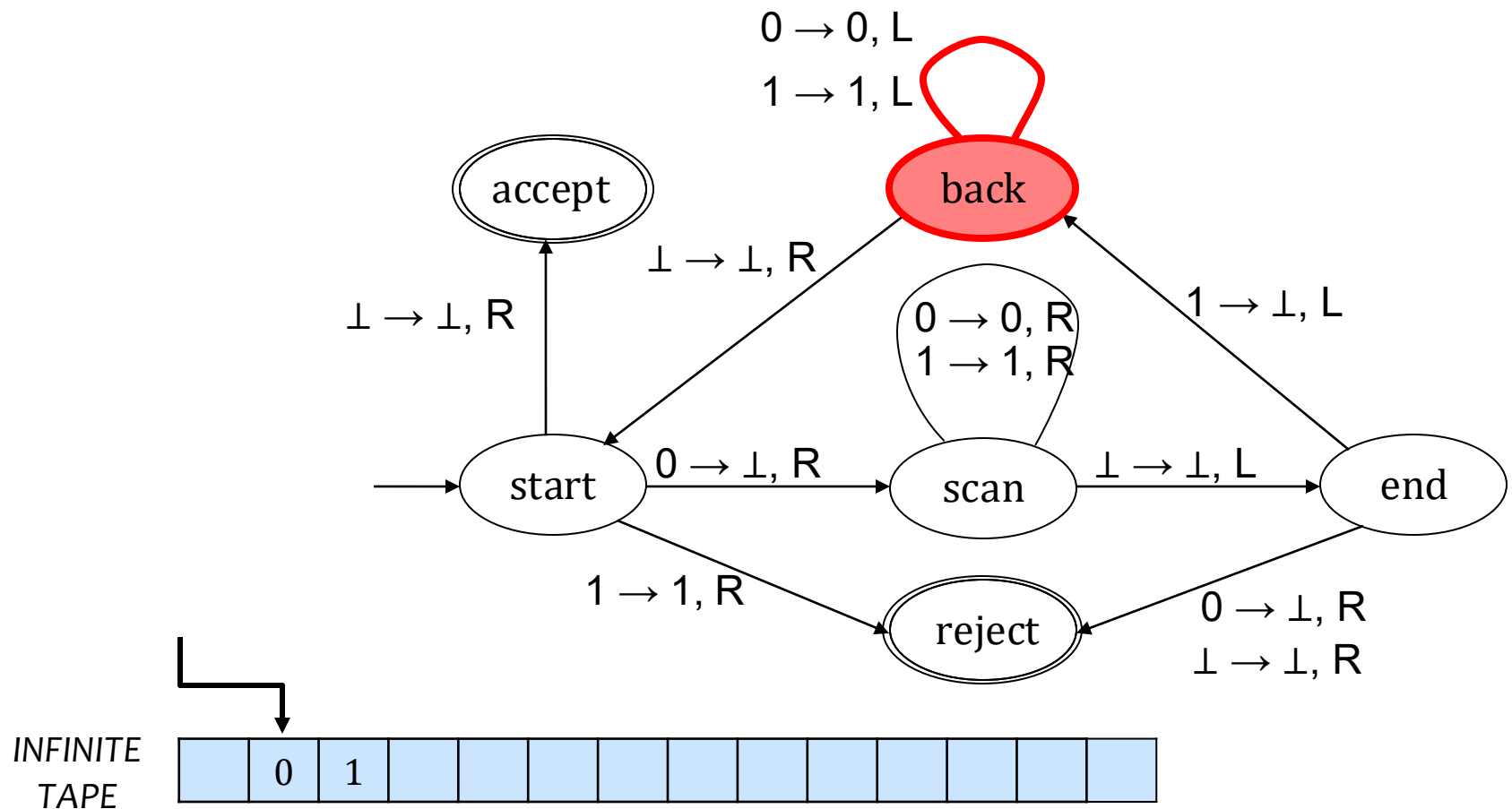




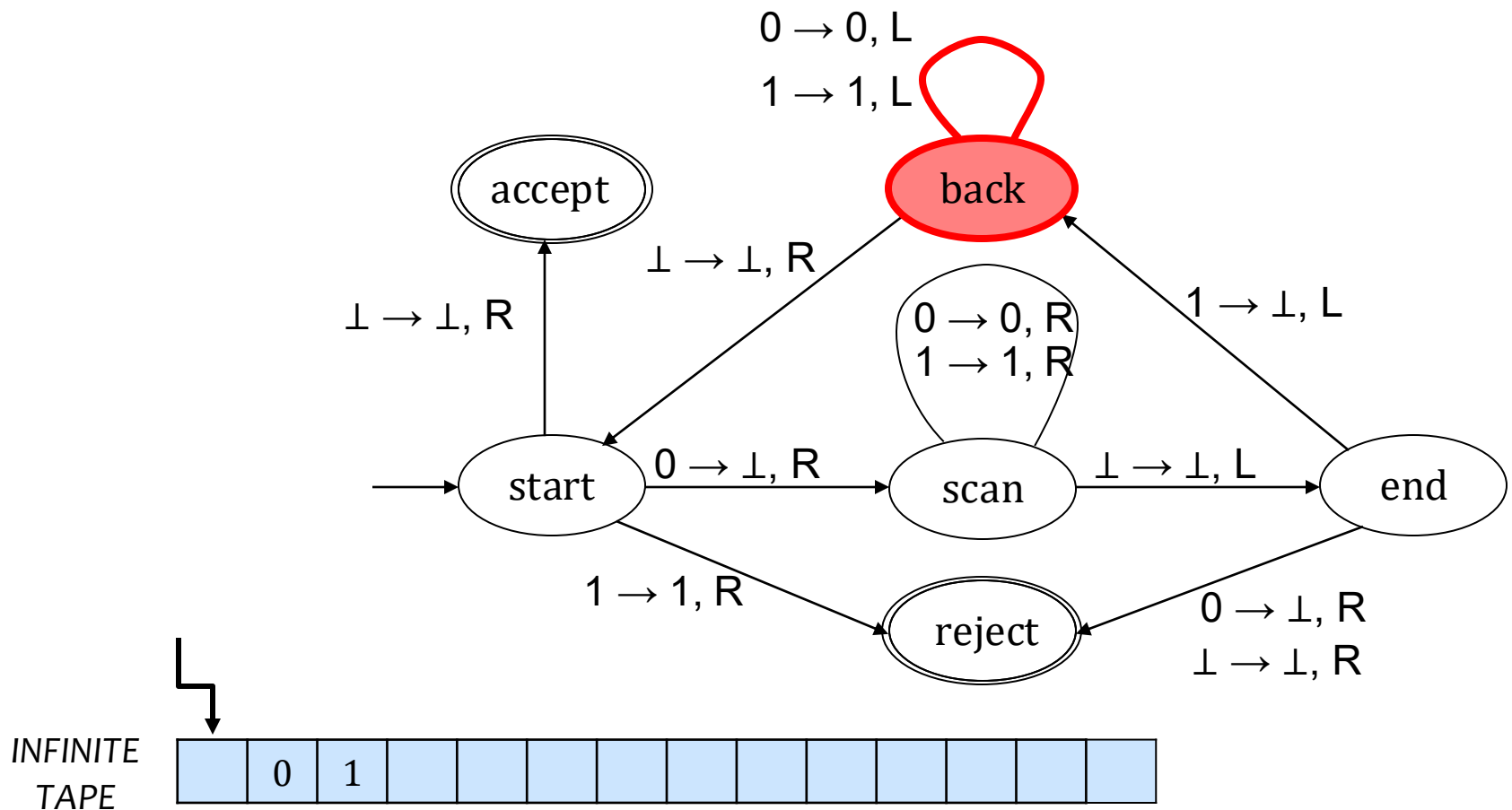
What is a Turing Machine?



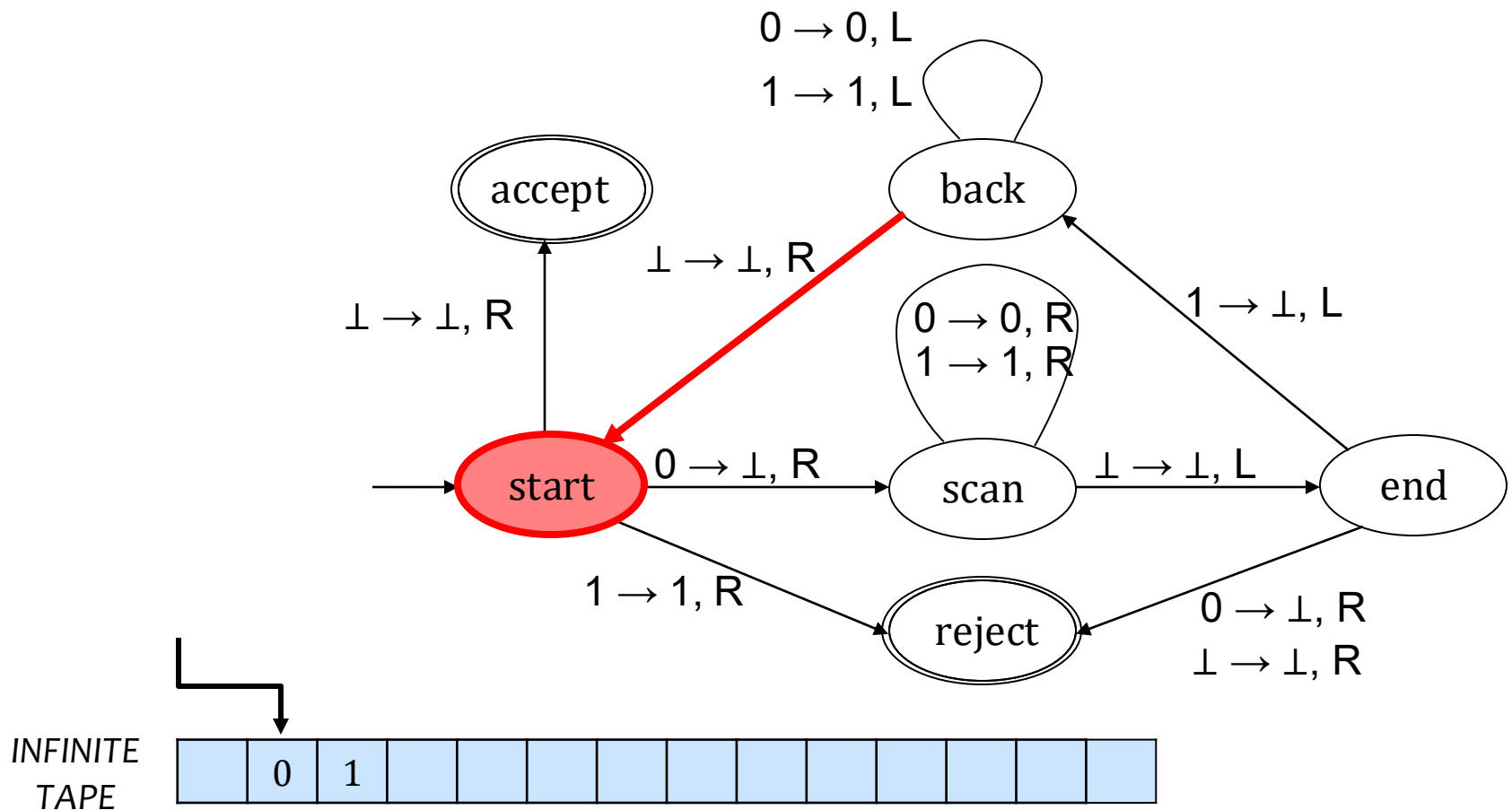
What is a Turing Machine?



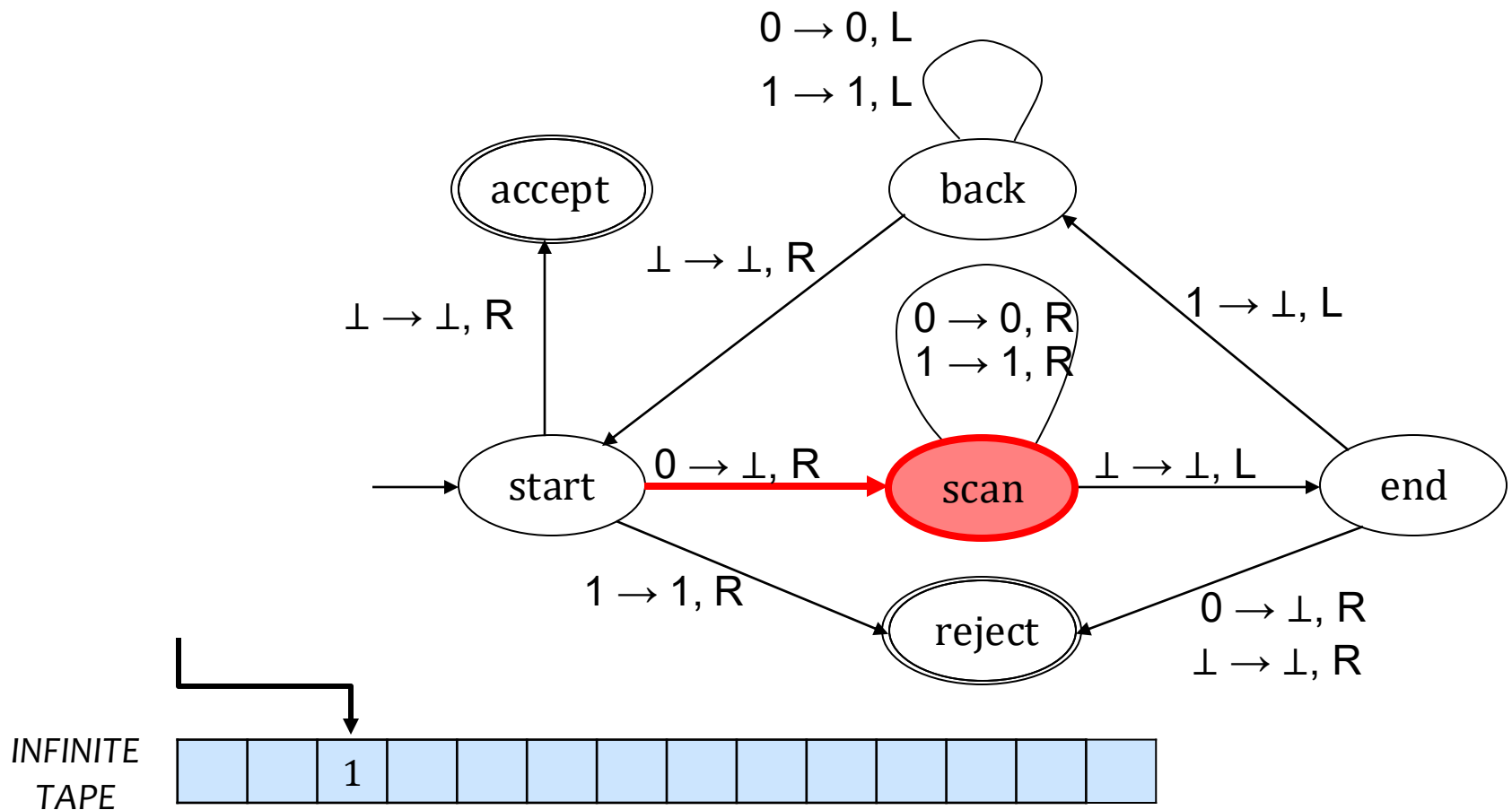
What is a Turing Machine?



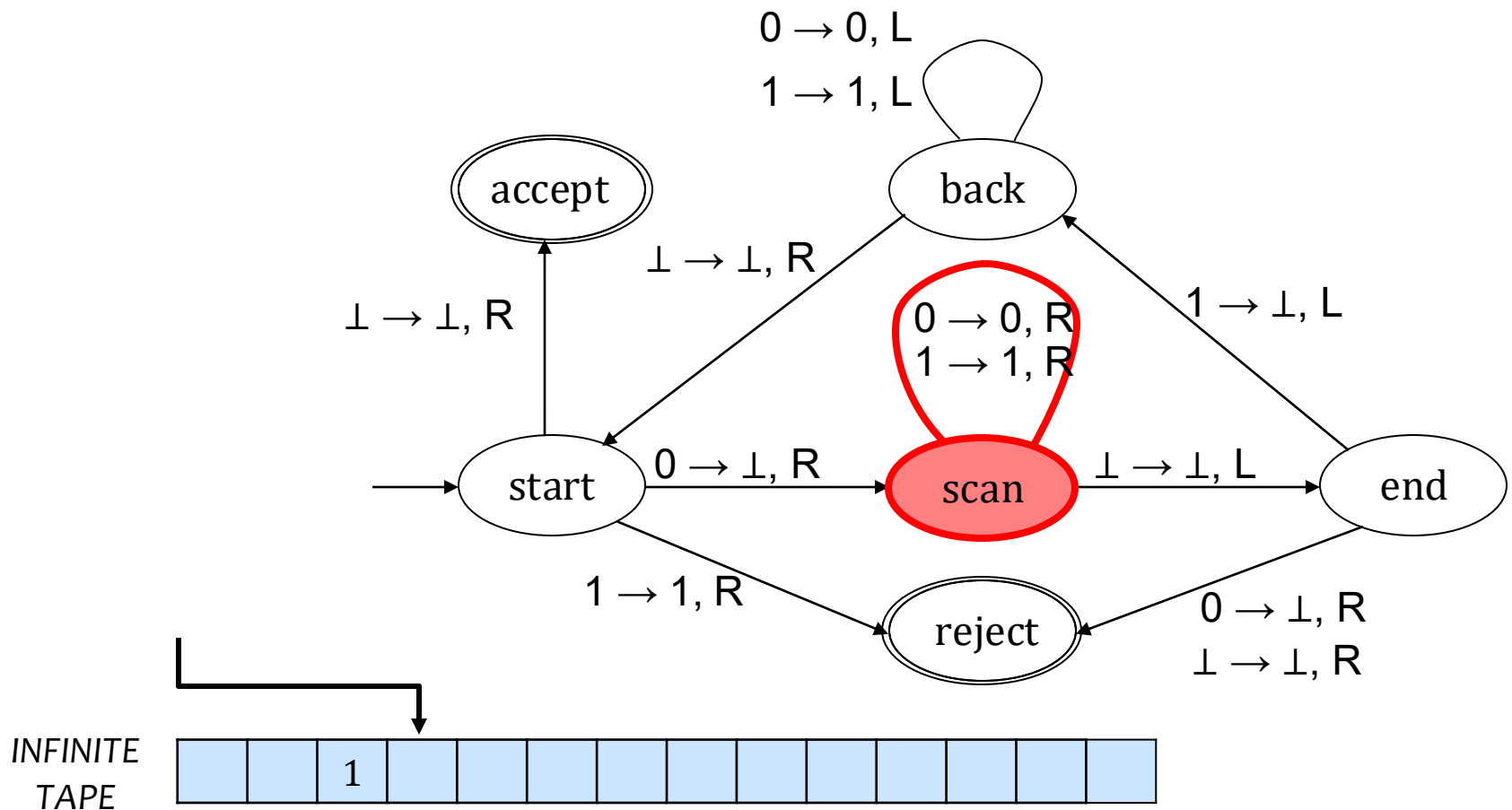
What is a Turing Machine?



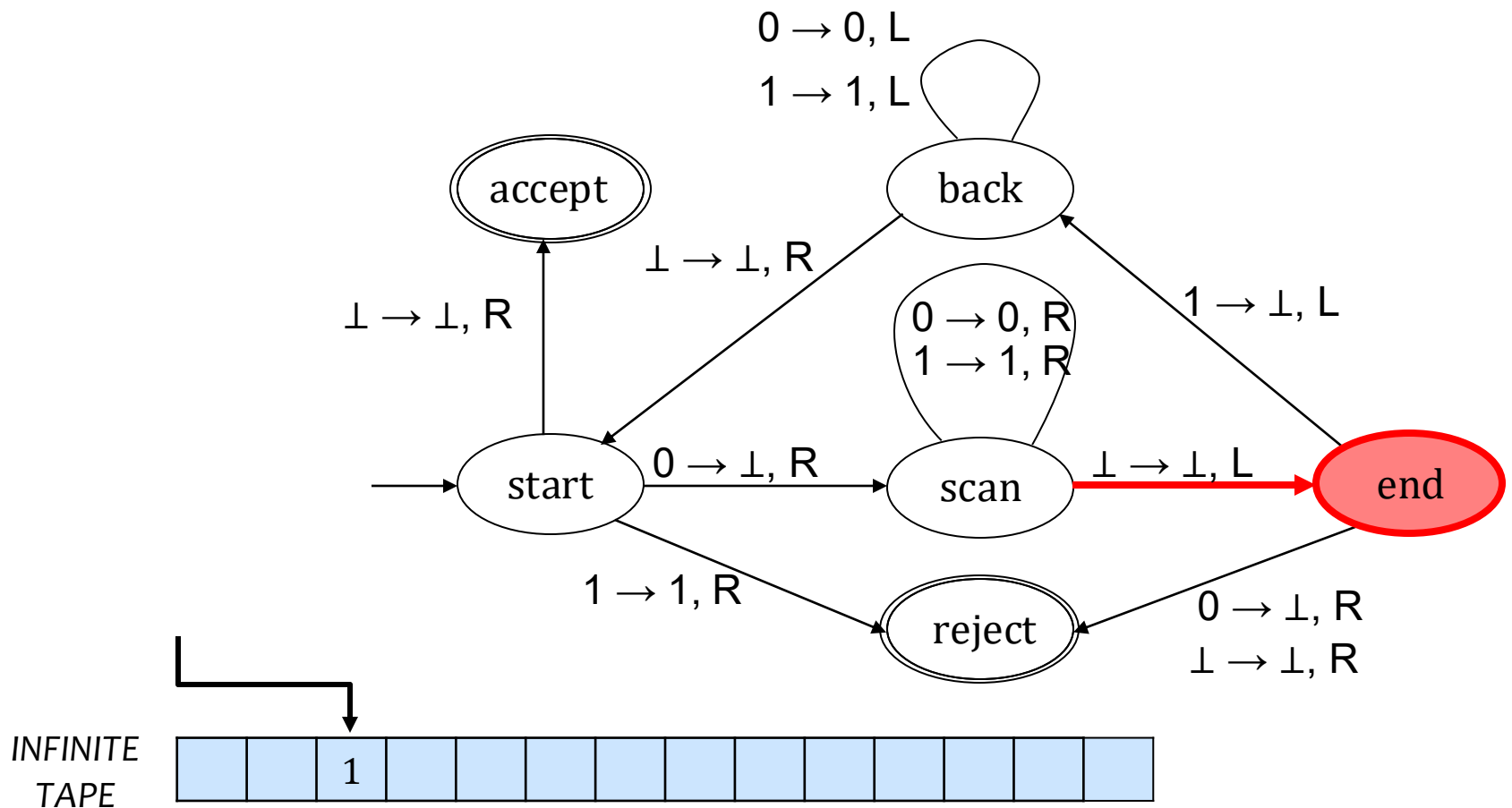
What is a Turing Machine?



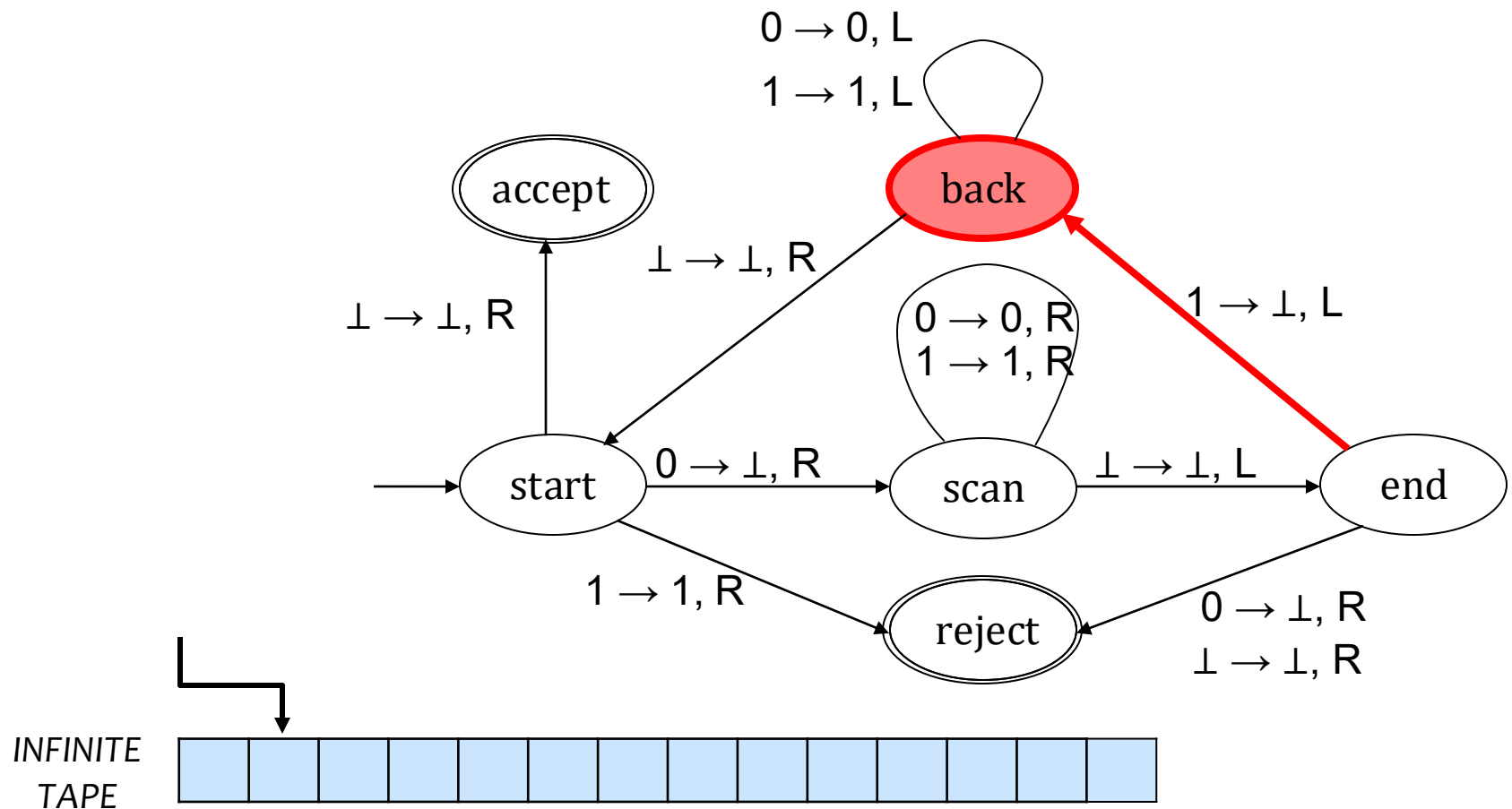
What is a Turing Machine?



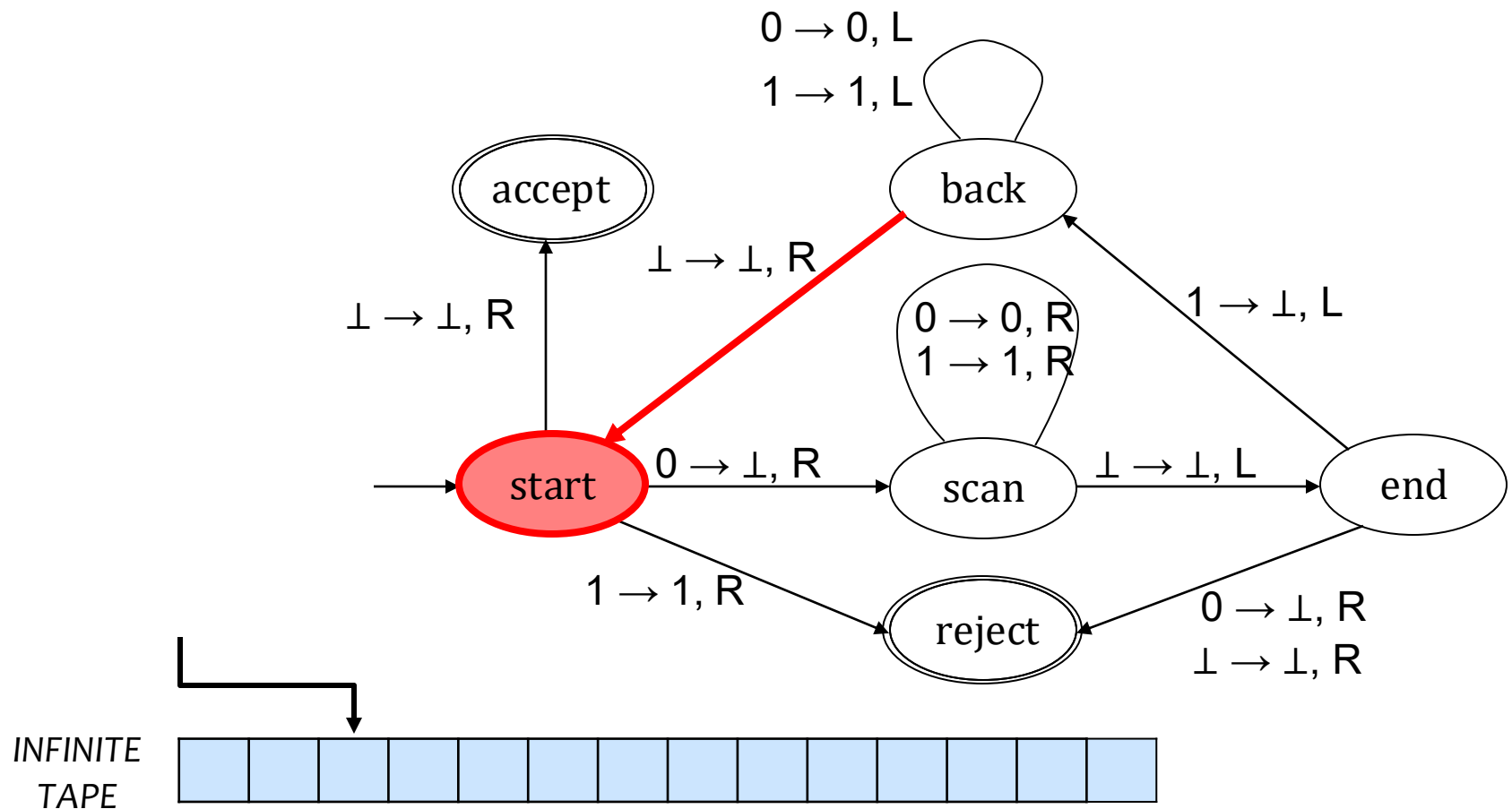
What is a Turing Machine?



What is a Turing Machine?



What is a Turing Machine?

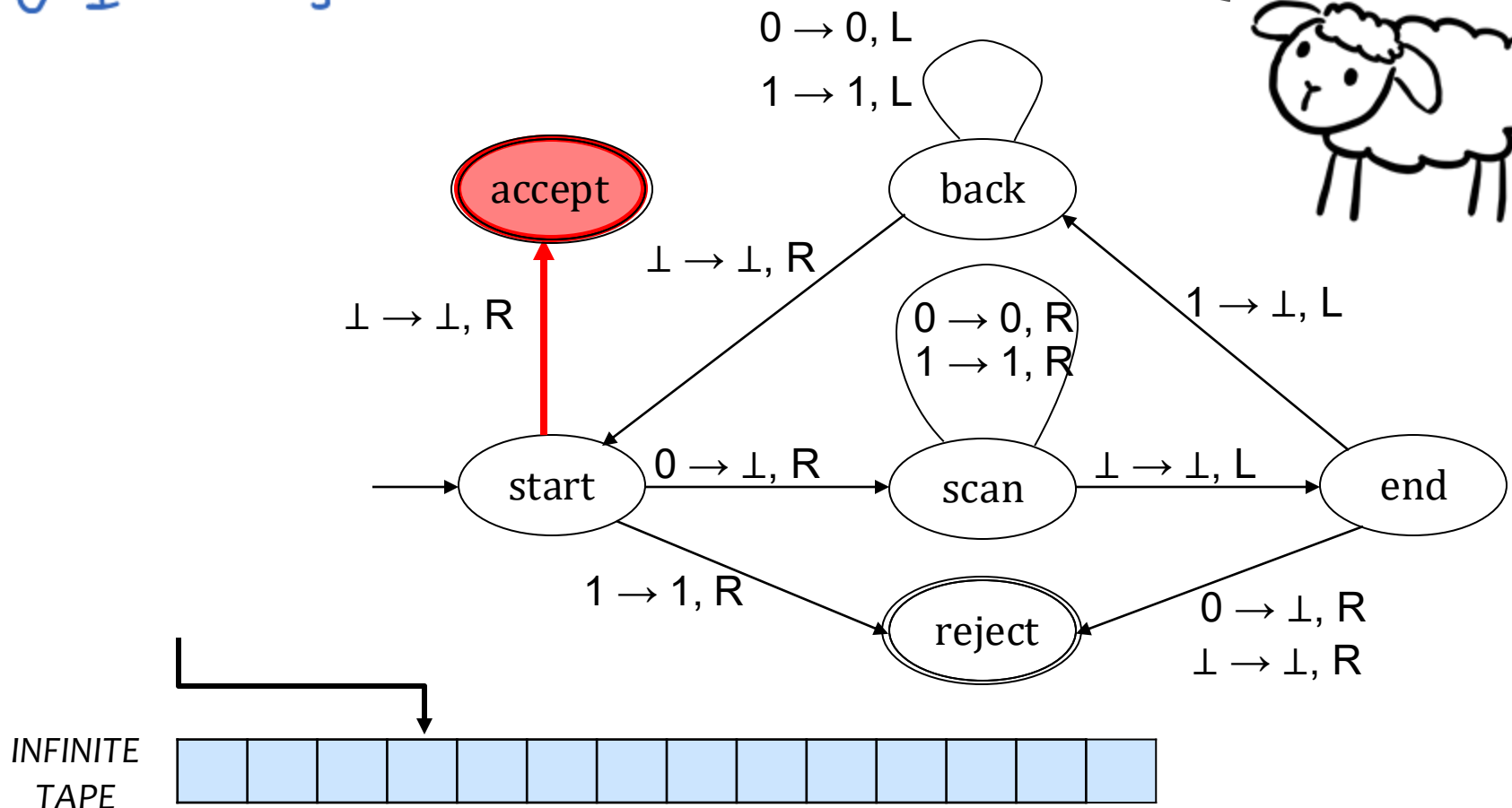


What is a Turing Machine?

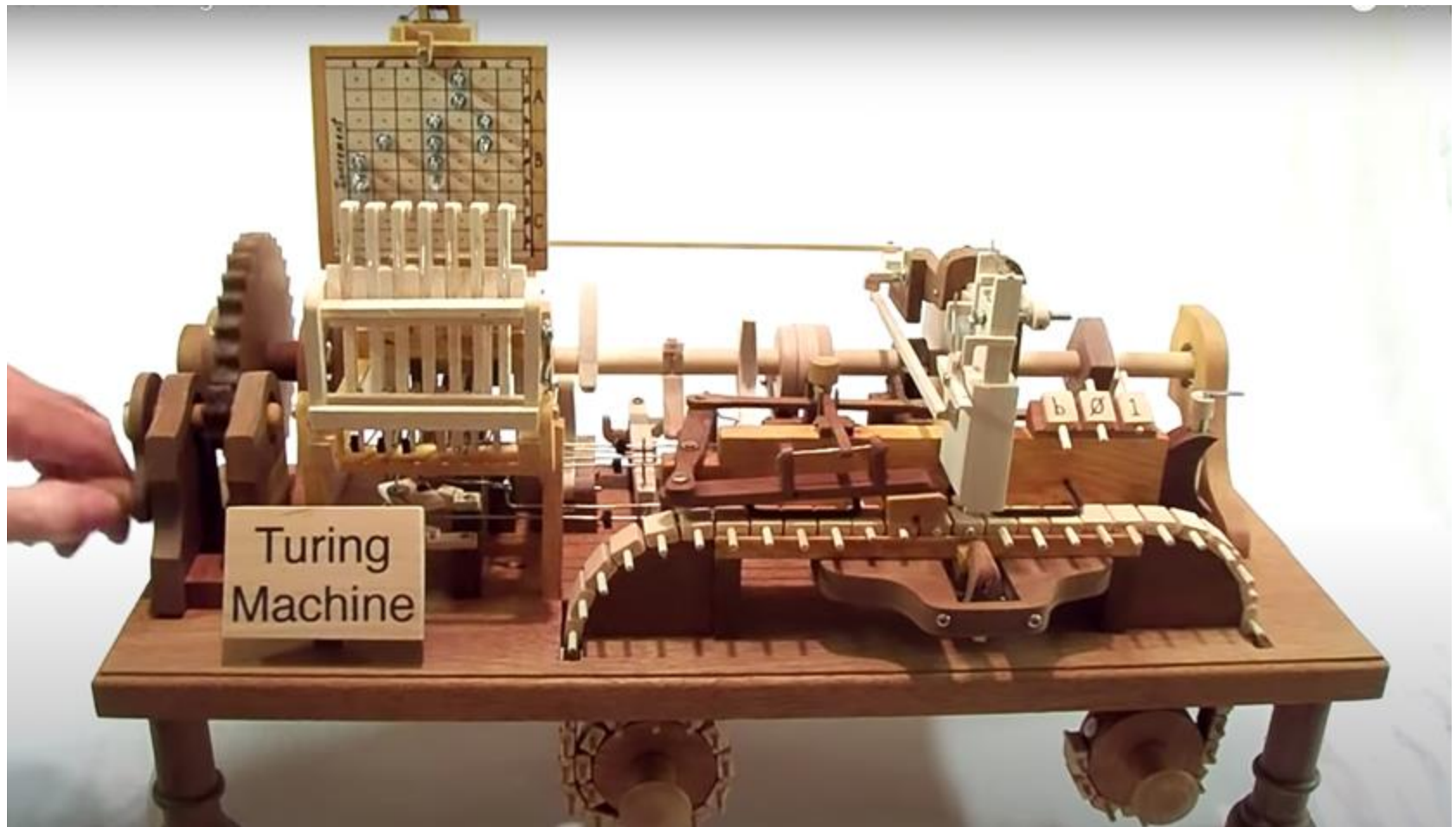
Q: Set of inputs accepted by this TM?

$\{0^k 1^k : k \geq 0\}$

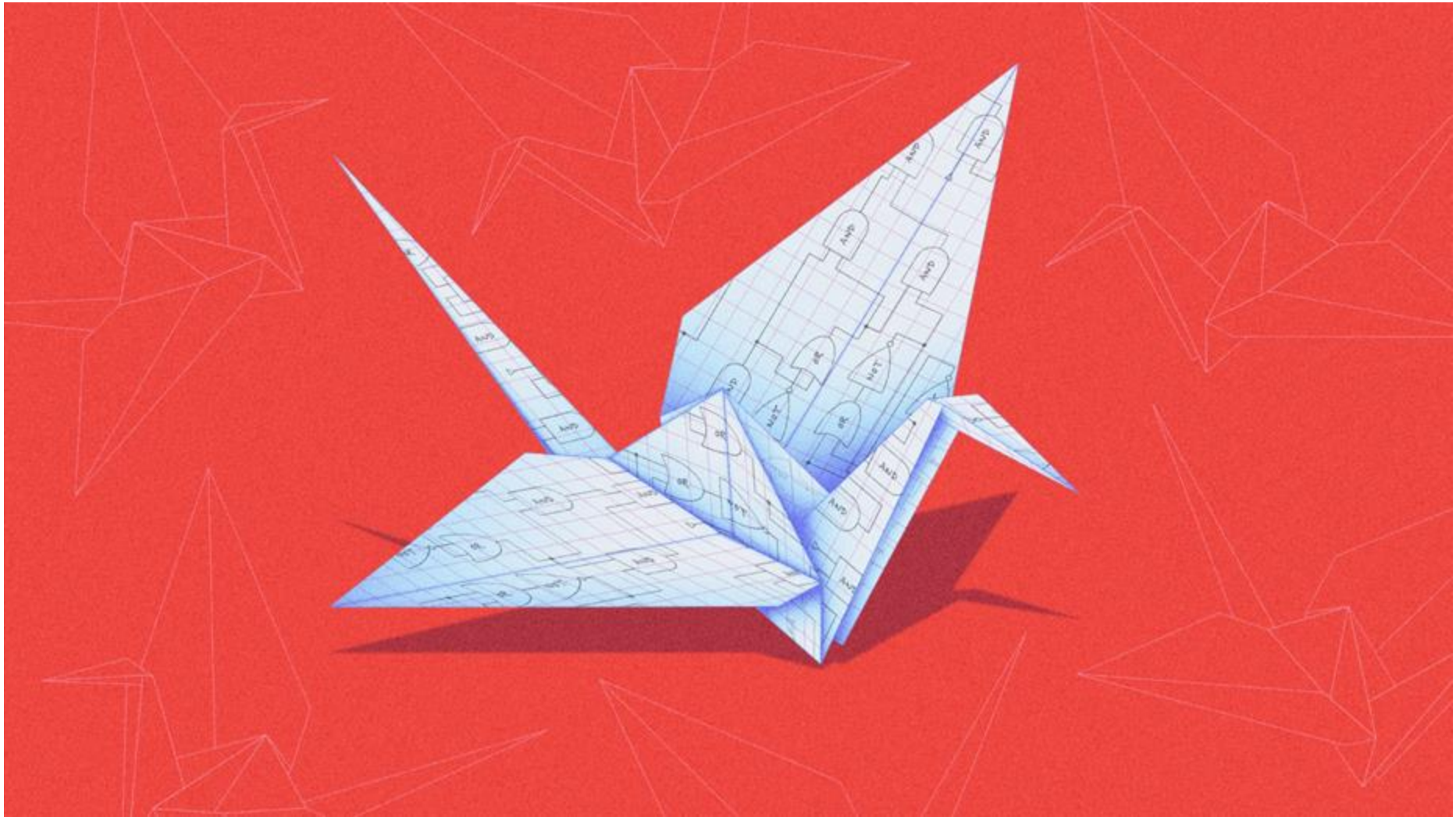
Whenever I program a Turing Machine I need to take a nap to recover



A Mechanical Turing Machine



Another Mechanical Turing Machine



[“How to Build an Origami Computer”](#), Quanta Magazine, 30 Jan 2024

Another Mechanical Turing Machine



[“Magic: The Gathering is Turing Complete”](#), arXiv:1904.09828

[“I Built a COMPUTER in Magic: The Gathering”](https://youtu.be/pdmODVYPDLA), <https://youtu.be/pdmODVYPDLA>

Formal Definition of a Turing Machine

A Turing machine is a seven-tuple

$$M = (\Sigma, \Gamma, Q, q_{\text{start}}, q_{\text{acc}}, q_{\text{rej}}, \delta)$$

- Σ is the (finite) *input alphabet*.
- Γ is the (finite) *tape alphabet*, $\Sigma \subseteq \Gamma$, contains *blank symbol* $\perp \in \Gamma$, $\perp \notin \Sigma$
- Q is the *finite set of states*.
- $q_{\text{start}} \in Q$ is the *initial state*.
- $q_{\text{acc}}, q_{\text{rej}} \in Q$ are the (distinct) *accept state* and *reject state*,
 $F = \{q_{\text{acc}}, q_{\text{rej}}\}$
- δ is the *transition function*.

All of these
sets are
finite



$$\delta: \underbrace{(Q \setminus F)}_{\text{Non-terminal state}} \times \underbrace{\Gamma}_{\text{Cell contents}} \rightarrow \underbrace{Q}_{\text{Next state}} \times \underbrace{\Gamma}_{\text{Symbol to write}} \times \underbrace{\{L, R\}}_{\text{Where to move the head}}$$

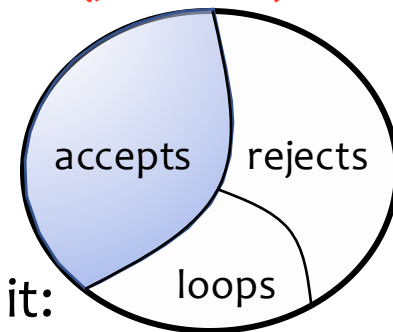
The Language of a TM

Question: What are the possible outcomes of a TM **M** on an input x ?

Answer: $M(x)$ either (i) accepts, (ii) rejects, or (iii) it “*loops*” (*forever*)

The language of a TM is the set of strings it accepts:

$$L(M) = \{x : M \text{ accepts } x\}$$



Definition: A Turing Machine **M** *recognizes* a language L if it:

1. accepts every string in L , and
2. rejects OR loops for every string not in L .

Definition: A Turing Machine **M** *decides* a language L if it:

1. accepts every string in L , and
2. rejects every string not in L (and does not loop on any input)

A language L is *decidable* if there is a TM that decides L .

Otherwise L is *undecidable*.

Fact: If a language **L** is decidable, then the **complement of L** is also decidable. Why?

Swap accept/reject states

Fact: If languages **L1** and **L2** are both decidable, then **L1 ∩ L2** and **L1 ∪ L2** are both decidable. Why?

M_1 decides L_1 , M_2 decides L_2

Define $M_A(x)$:

Run $M_1(x)$ and $M_2(x)$,
if both accept: accept
else: reject

$M_U(x)$:

Run $M_1(x)$ and $M_2(x)$
if either accepts: accept
else: reject

$x \in L_1 \cap L_2 \Rightarrow M_1(x)$ and $M_2(x)$ both accept
 $\Rightarrow M_A(x)$ accepts

Want to show: $x \in L_1 \cap L_2 \Rightarrow M_A(x)$ accepts

$x \notin L_1 \cap L_2 \Rightarrow M_A(x)$ rejects
 $\Rightarrow$ at least one of $M_1(x)$ or $M_2(x)$ rejects.
 $\Rightarrow M_A(x)$ rejects.

In these proofs we can use pseudocode instead of specifying a TM, since pseudocode has the same computational power as a TM!



What if we added more tapes to a TM?

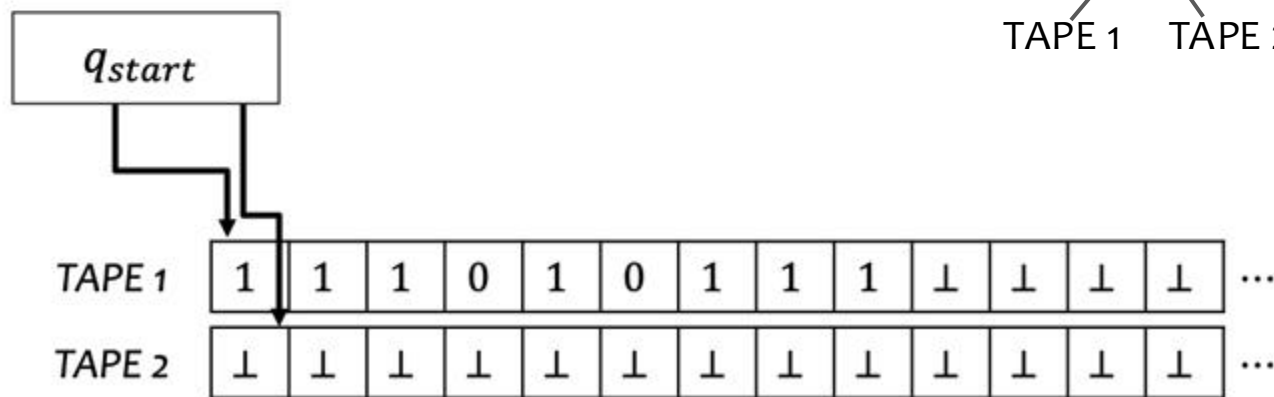
Well, the Church-Turing Thesis says there's no “computer” more powerful than a 1-tape TM.

So a 2-tape TM better not be more powerful.

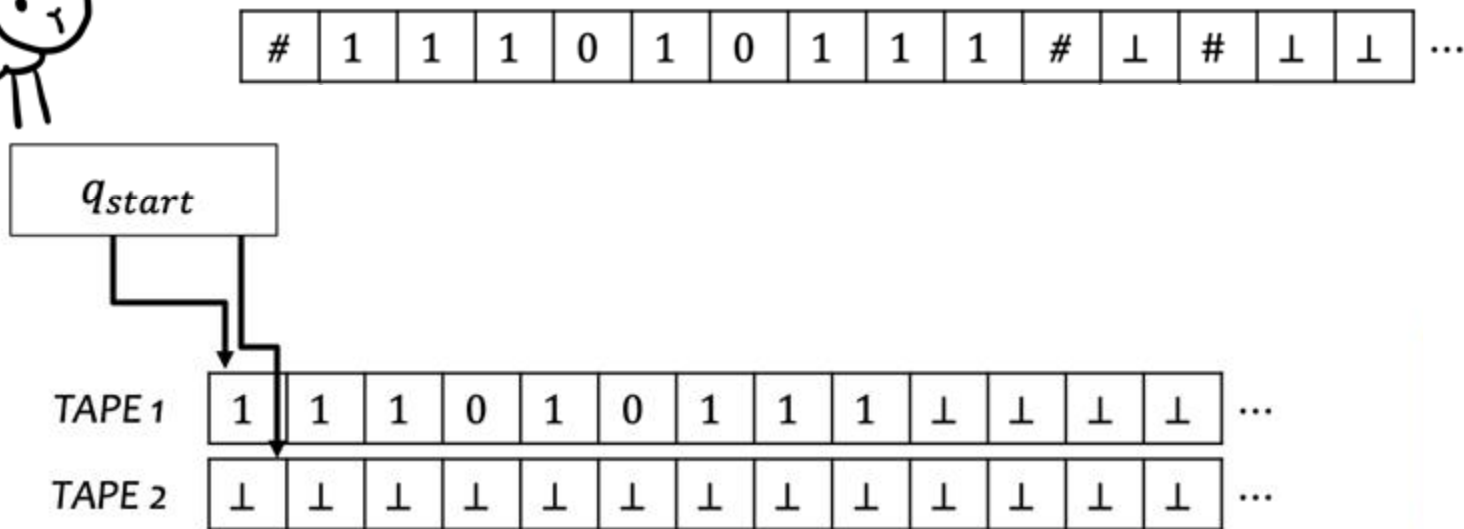
Let's prove it!

Example transition rule for 2-tape TM: $\delta(q_1, 1, \perp) = (q_2, 0, 1, R, \dot{L})$

Diagram illustrating the transition rule: The input symbol '1' is on TAPE 1, and the blank symbol ' $\perp$ ' is on TAPE 2. The rule specifies that the machine transitions to state q_2 , writes '0' on TAPE 1, writes '1' on TAPE 2, moves the head on TAPE 1 to the right (R), and moves the head on TAPE 2 to the left (L).



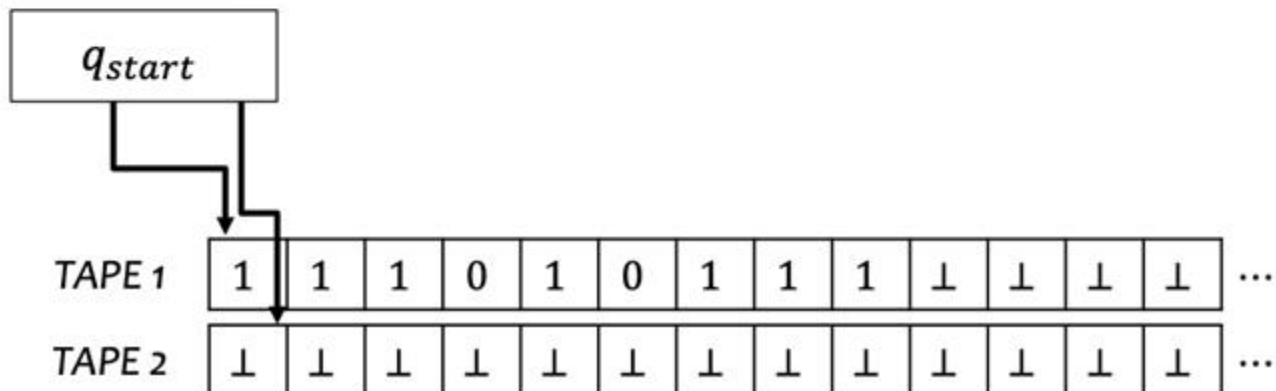
Similar reasoning shows that a TM with a 1-way infinite tape is equivalent to a TM with a 2-way infinite tape (used in turingmachine.io)



Equivalence of 1-tape and 2-tape TMs

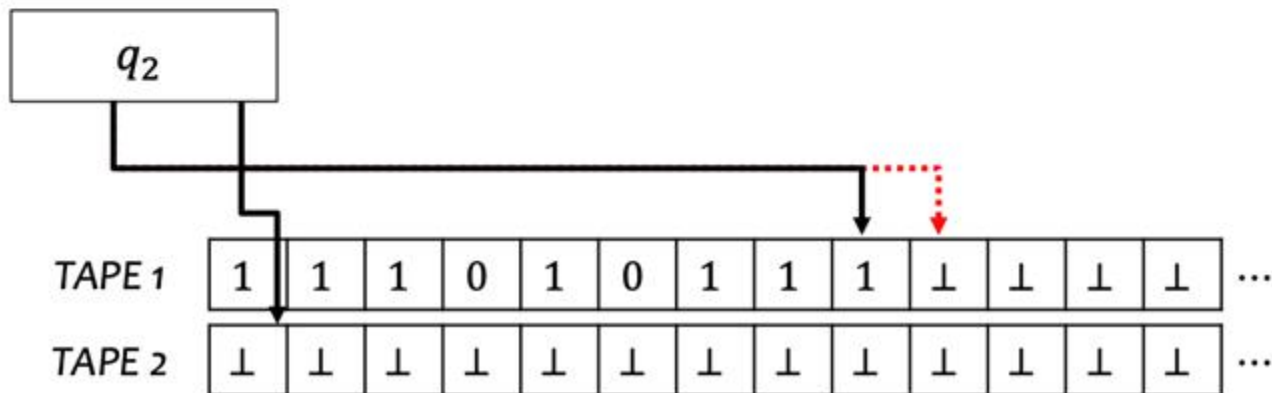
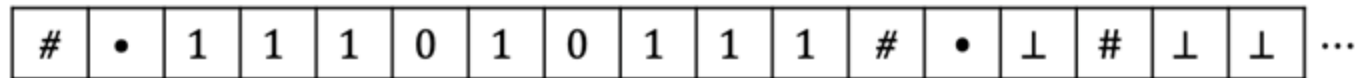
Idea: Need to demarcate location of each head – use a • symbol to the left of the cell where the head is.

#	•	1	1	1	0	1	0	1	1	1	#	•	⊥	#	⊥	⊥	...
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	-----



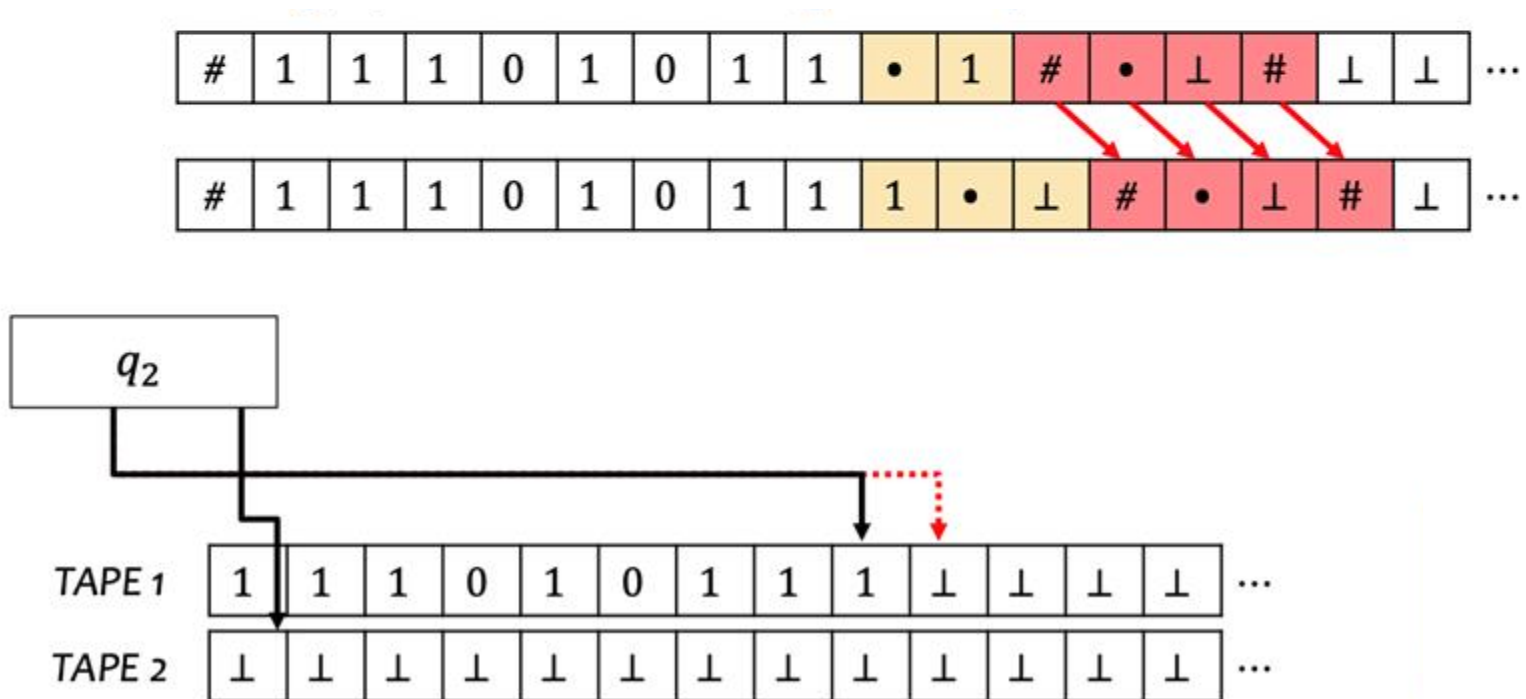
Equivalence of 1-tape and 2-tape TMs

Issue 1: What if we need to insert a symbol at the end of TAPE 1 and there's no room?



Equivalence of 1-tape and 2-tape TMs

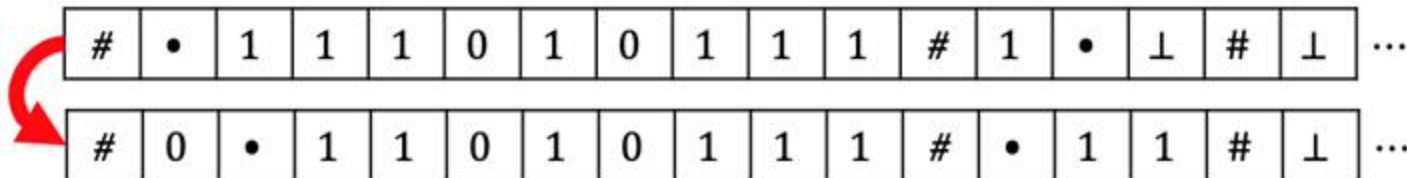
Issue 1: What if we need to insert a symbol at the end of TAPE 1 and there's no room?



Equivalence of 1-tape and 2-tape TMs

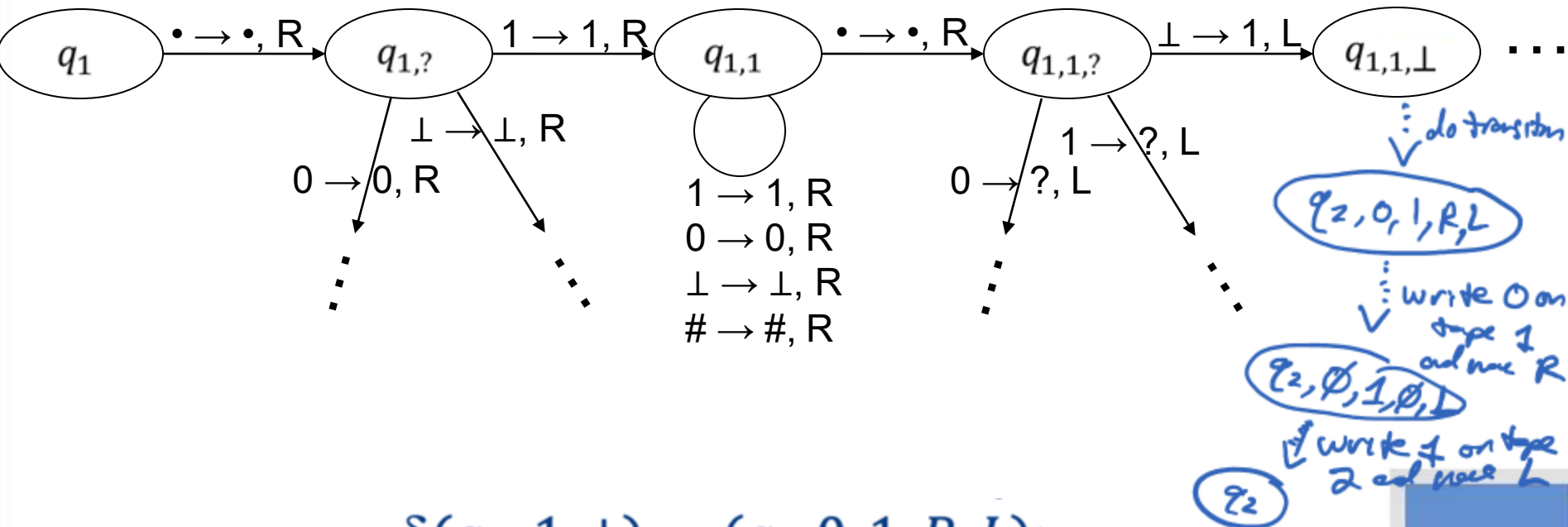
Issue 2: To know which transition rule to use, I need to know the current symbols of **both** tapes of the 2-tape TM, but I only have 1 head on my 1-tape TM.

$$\delta(q_1, 1, \perp) = (q_2, 0, 1, R, L):$$

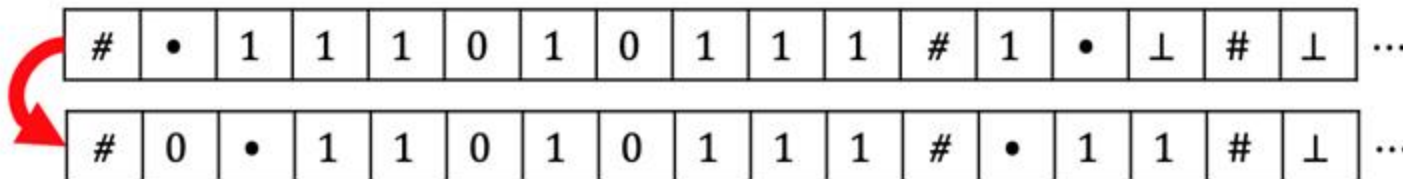


Equivalence of 1-tape and 2-tape TMs

Issue 2: To know which transition rule to use, I need to know the current symbols of **both** tapes of the 2-tape TM, but I only have 1 head on my 1-tape TM.



$$\delta(q_1, 1, \perp) = (q_2, 0, 1, R, L):$$



HOW TO
MOVE TO A
DIFFERENT
STATE

